

SALESADMIN AND EMPLOYEE APPLICATION - OVERALL STRUCTURE

1. APPLICATION OVERVIEW

Application Name: Sales Connect / Employee Connect

Purpose: Enterprise Sales Force Automation and Management System

Core Functionality: Sales team management, journey planning, dealer management, order processing, performance tracking, and analytics

2. APPLICATION ARCHITECTURE

2.1 Frontend Structure

text

frontend/

```
├── src/
│   ├── components/    # Reusable UI components
│   │   ├── Sidebar.tsx # Main navigation (current file)
│   │   ├── Header.tsx  # Top header bar
│   │   ├── common/     # Shared components
│   │   └── forms/      # Form components
│   ├── pages/         # Main application pages
│   │   ├── Dashboard.tsx # Main dashboard
│   │   ├── employees/   # Employee management
│   │   ├── dealers/    # Dealer management
│   │   ├── orders/     # Order management
│   │   ├── journeyplan/ # Journey planning
│   │   ├── tasks/      # Task management
│   │   ├── attendance/  # Attendance tracking
│   │   ├── analytics/   # Analytics dashboard
│   │   ├── leads/      # Lead management
│   │   ├── incentives/  # Incentive management
│   │   ├── communication/ # Communication center
│   │   ├── settings/   # System settings
│   │   └── performance/ # Performance modules
│   ├── layouts/       # Layout wrappers
│   ├── hooks/         # Custom React hooks
│   ├── utils/         # Utility functions
│   ├── config/        # Configuration files
│   └── types/         # TypeScript type definitions
```

2.2 Backend Structure

text

backend/

```
├── project/
│   ├── models/          # Database models
│   │   ├── user_models.py    # User/Employee models
│   │   ├── dealer_models.py  # Dealer models
│   │   ├── order_models.py   # Order models
│   │   ├── journey_models.py # Journey plan models
│   │   ├── task_models.py    # Task models
│   │   ├── attendance_models.py # Attendance models
│   │   ├── lead_models.py    # Lead management models
│   │   └── incentive_models.py # Incentive models
│   ├── routes/           # API endpoints
│   │   ├── auth_routes.py    # Authentication routes
│   │   ├── user_routes.py    # User management routes
│   │   ├── dealer_routes.py   # Dealer management routes
│   │   ├── order_routes.py    # Order management routes
│   │   └── dashboard_routes.py # Dashboard data routes
│   ├── services/         # Business logic
│   ├── utils/            # Utility functions
│   ├── config/           # Configuration
│   └── app.py            # Main application entry
```

3. USER ROLES AND PERMISSIONS

Based on the Sidebar navigation, the application supports 5 main user types:

3.1 Admin/Super Admin (base_location)

- **Access Level:** Full system access
- **Can Access:** All features including administrative dashboards
- **Special Features:** Admin Dashboard, Zone/State Performance dashboards

3.2 Zone Manager

- **Access Level:** Zone-level management
- **Can Access:** Employee management, dealer management, zone analytics
- **Restricted From:** Admin-specific dashboards

3.3 State Manager

- **Access Level:** State-level management
- **Can Access:** State analytics, dealer management, team performance
- **Restricted From:** Employee management (some features)

3.4 Territory Manager

- **Access Level:** Territory-level operations
- **Can Access:** Journey planning, dealer visits, task management
- **Restricted From:** Higher-level analytics

3.5 Area Employee

- **Access Level:** Field employee basic access
- **Can Access:** Personal journey plan, my tasks, my performance
- **Restricted From:** Management features, team analytics

4. MODULE CATEGORIES

4.1 Core Management Modules

1. **Employee Management** - User/team management and hierarchy
2. **Dealer Management** - Dealer/retailer management
3. **Order Management** - Sales order processing
4. **Catalogue Management** - Product catalogue

4.2 Field Operations Modules

1. **Journey Plan Management** - Route planning and scheduling
2. **Task Management** - Task assignment and tracking
3. **Attendance Tracking** - Employee attendance monitoring
4. **Lead Management** - Sales lead tracking

4.3 Analytics & Performance Modules

1. **Performance Dashboards** - Individual and team performance
2. **Analytics** - Sales and operational analytics
3. **Leaderboard** - Competitive performance ranking

4. **Incentive Management** - Reward and incentive tracking

4.4 Support & Communication Modules

1. **Communication** - Internal messaging
2. **Support Tickets** - Issue tracking system
3. **Notifications** - System notifications

5. TECHNOLOGY STACK

Frontend Stack

- **Framework:** React 18+ with TypeScript
- **Routing:** Wouter (lightweight router)
- **State Management:** React Query (server state), React Context (client state)
- **UI Components:** Custom with Tailwind CSS
- **Icons:** Lucide React
- **Forms:** React Hook Form (inferred from common patterns)
- **Validation:** Zod or Yup (inferred)
- **HTTP Client:** Fetch API with custom wrappers

Backend Stack

- **Framework:** Flask (Python)
- **ORM:** SQLAlchemy
- **Database:** PostgreSQL (production), SQLite (development)
- **Authentication:** JWT (JSON Web Tokens)
- **API Design:** RESTful architecture
- **File Storage:** Local/S3 for avatars and attachments

6. KEY FEATURES IDENTIFIED FROM NAVIGATION

1. **Role-Based Access Control (RBAC)** - Dynamic navigation based on user role
2. **Multi-Level Dashboard** - Different dashboards for different user levels
3. **Performance Tracking** - Individual and team performance metrics

4. **Field Force Automation** - Journey planning and task management
5. **Sales Operations** - Order processing, dealer management, lead tracking
6. **Communication System** - Integrated messaging and notifications
7. **Analytics Suite** - Comprehensive sales and operational analytics
8. **Incentive Management** - Performance-based reward system

7. DATA FLOW OVERVIEW

1. Authentication Flow:

- a. User logs in → JWT token generated → Stored in session Storage
- b. Token used for all subsequent API calls
- c. User data cached client-side

2. Navigation Flow:

- a. App loads → Check user role → Filter navigation items → Render sidebar
- b. User clicks item → Route changes → Load corresponding page module

3. Data Management Flow:

- a. Page loads → React Query fetches data → Renders UI
- b. User actions → API calls → Backend processes → UI updates
- c. Real-time updates via WebSocket or polling (inferred)

8. SECURITY MODEL

1. **Authentication:** JWT-based with session storage
2. **Authorization:** Role-based + location-based permissions
3. **Data Isolation:** Users only see data within their location hierarchy
4. **API Protection:** All endpoints validate JWT and permissions
5. **Client-Side Security:** Navigation filtering + backend validation

9. SCALABILITY CONSIDERATIONS

1. **Modular Architecture:** Each feature module is independent

2. **Role-Based Scaling:** Permission system supports organizational growth
3. **Database Design:** Hierarchical location structure supports multi-level organizations
4. **API Design:** RESTful endpoints with pagination, filtering, sorting
5. **Caching Strategy:** React Query for efficient data fetching

10. DOCUMENTATION APPROACH

Based on the provided format, we will document each module/page separately following this structure:

1. **Module Name & Purpose** - What the module does and who uses it
2. **Entry Points** - Frontend and backend entry points
3. **Files Involved** - All related code files
4. **Tools Used** - Technologies and libraries
5. **Database Tables** - Relevant database schema
6. **API Endpoints** - REST endpoints for the module
7. **Working Procedure** - Step-by-step user flows
8. **Error Handling** - How errors are managed
9. **Non-Functional Requirements** - Performance, security, scalability
10. **Final Outcome** - Summary of what the module achieves

MODULES INVOLVED

Based on the navigation items in the Sidebar, we should document these modules in order:

1. **Dashboard Module** (Main dashboard page)
2. **Employee Management Module**
3. **Hierarchy Module**
4. **Journey Plan Management Module**
5. **Dealer Management Module**
6. **Order Management Module**
7. **Task Management Module**

8. **Attendance Module**
9. **Analytics Module**
10. **Lead Management Module**
11. **Incentive Management Module**
12. **Performance Modules** (My Performance, Team Performance, Leaderboard)
13. **Communication Module**
14. **Settings Module**

DASHBOARD MODULE

1. DASHBOARD PURPOSE

The Dashboard module serves as the **centralized performance monitoring and management hub** for the Sales Admin application. It provides role-based, hierarchical views of sales performance across the entire organization, from individual field employees to regional managers and administrators. The module solves the problem of **unified performance visibility** by aggregating data from multiple sources (sales, targets, orders, activities) and presenting it through interactive visualizations and actionable insights.

Primary Users:

- **Admin/Super Admin** (base_location): Full organization-wide performance visibility with hierarchical breakdowns
- **Zone Managers**: Zone-level performance monitoring with state-wise comparisons
- **State Managers**: State-level analytics with territory performance tracking
- **Territory Managers**: Territory-focused performance with area-level insights
- **Area Employees**: Personal performance dashboard with individual metrics

2. ENTRY POINT

Frontend Entry File:

- frontend/pages/Dashboard.tsx (Main dashboard page)
- frontend/components/dashboard/ (All dashboard visualization components)

Backend API Base Path:

- /api/performance/admin-dashboard (Primary dashboard data endpoint)
- /api/performance/my (Individual performance data)
- /api/ai/insights (AI-powered insights and recommendations)

3. FILES INVOLVED

Frontend:

- Dashboard.tsx (Main dashboard page component)
- components/dashboard/metrics-cards.tsx (Summary metrics cards)
- components/dashboard/sales-chart.tsx (Sales trend visualization)
- components/dashboard/performance-distribution.tsx (Performance classification chart)
- components/dashboard/zone-achievementcomparision.tsx (Zone comparison panel)
- components/dashboard/hierarchy-visualization.tsx (Organizational hierarchy view)
- components/dashboard/recent-activities.tsx (Activity feed)
- components/dashboard/top-performers.tsx (Leaderboard view)
- lib/queryClient.ts (React Query hooks for data fetching)

Backend:

- backend/project/routes/performance_routes.py (Performance API endpoints)
- backend/project/routes/ai_insight_routes.py (AI insights endpoints)
- backend/project/services/sales_performance_service.py (Performance calculation logic)
- backend/project/services/ai_insight/engine.py (AI insight generation engine)
- backend/project/models/ (User, Employee, Location, Target models)
- backend/project/models/ai_insights.py (AI insight logging model)

4. TOOLS USED

Frontend:

- React 18+ with TypeScript
- React Query (TanStack Query) for data fetching and caching

- Wouter for client-side routing
- date-fns for date formatting
- Lucide React for icons
- Tailwind CSS for styling
- Custom UI components (Card, Button, etc.)

Backend:

- Flask with Flask-JWT-Extended
- SQLAlchemy ORM for database operations
- PostgreSQL for primary data storage
- Custom AI insight engine with rule-based analysis

5. DATABASE TABLE(S)

Primary Tables Used:

Table Name: employees

Columns:

- id (UUID, Primary Key)
- first_name (String)
- last_name (String)
- email (String, Unique)
- role (String)
- created_at (Timestamp)
- updated_at (Timestamp)

Table Name: locations

Columns:

- id (UUID, Primary Key)
- name (String)
- location_type (Enum: zone, state, territory, area)
- parent_id (UUID, Foreign Key to locations.id)
- created_at (Timestamp)
- updated_at (Timestamp)

Table Name: employee_locations

Columns:

- id (UUID, Primary Key)
- employee_id (UUID, Foreign Key to employees.id)
- location_id (UUID, Foreign Key to locations.id)
- created_at (Timestamp)

Table Name: sales_targets

Columns:

- id (UUID, Primary Key)
- salesperson_id (UUID, Foreign Key to employees.id)
- month (String, YYYY-MM format)
- target_amount (Decimal)
- created_at (Timestamp)
- updated_at (Timestamp)

Table Name: insight_logs

Columns:

- id (UUID, Primary Key)
- user_id (UUID, Foreign Key to users.id)
- insight_code (String)
- payload (JSON)
- severity (String)
- created_at (Timestamp)

6. API ENDPOINTS

Dashboard Data:

- GET /api/performance/admin-dashboard?month=YYYY-MM - Complete dashboard data (scope-aware)
- GET /api/performance/summary?month=YYYY-MM - Performance summary
- GET /api/performance/state?month=YYYY-MM&state_id=UUID - State-level performance

- GET /api/performance/zone?month=YYYY-MM&zone_id=UUID - Zone-level performance
- GET /api/performance/locations - Accessible locations for user

AI Insights:

- GET /api/ai/insights - AI-powered insights and recommendations

Additional Performance Endpoints:

- GET /api/performance/my?month=YYYY-MM - Individual performance
- GET /api/performance/trend?months=6 - Monthly trend data
- GET /api/performance/leaderboard?month=YYYY-MM - Leaderboard rankings
- GET /api/performance/export?month=YYYY-MM&format=csv - Export performance data

7. DASHBOARD PAGE WORKING PROCEDURE

7.1 PAGE LOAD AND INITIALIZATION

1. User navigates to /dashboard from sidebar
2. Dashboard component mounts and retrieves user data from sessionStorage
3. Current month is set as default (format: YYYY-MM)
4. Role-based visibility configuration loads
5. React Query fetches dashboard data from /api/performance/admin-dashboard
6. User's location type determines data scope (organization, zone, state, territory, area)

7.2 SCOPE-AWARE DATA FETCHING

1. Backend determines user's scope based on role and location assignments:
 - a. **Admin:** Full organizational data with hierarchical breakdown
 - b. **Zone Manager:** Zone-scoped data with state-level details
 - c. **State Manager:** State-scoped data with territory-level details
 - d. **Territory Manager:** Territory-scoped data with area-level details
 - e. **Area Employee:** Individual performance data only
2. Data includes: summary metrics, trend data, performance distribution, hierarchy visualization

7.3 COMPONENT RENDERING PIPELINE

1. **MetricsCards:** Displays summary statistics (total sales, target, achievement, employees)
2. **SalesChart:** Shows monthly sales trend and zone-wise performance comparison
3. **Quick Actions Panel:** Role-specific actions filtered by user permissions
4. **PerformanceDistribution:** Visualizes performance classification (exceeded, on-track, etc.)
5. **AllInsightPanel:** Displays AI-generated insights and recommendations
6. **OrganizationalHierarchy:** Interactive hierarchical view of performance across locations
7. **TopPerformers:** Leaderboard showing top performers within scope

7.4 QUICK ACTIONS NAVIGATION

1. Quick actions are filtered based on user's allowed routes from visibilityConfig
2. Two types of actions:
 - a. **Route Navigation:** Navigates to other pages (journey plan, tasks, analytics)
 - b. **Modal Actions:** Opens modals (Add Employee modal)
3. Clicking an action either navigates or opens corresponding modal

7.5 MONTH SELECTION AND DATA REFRESH

1. Month selector dropdown shows last 12 months
2. Changing month triggers new API call with updated month parameter
3. All dashboard components re-render with new month's data
4. AI insights are regenerated for the selected month

7.6 AI INSIGHTS GENERATION

1. Backend AI engine analyzes user's performance data
2. Rule-based insights are generated based on user role and performance patterns
3. Insights are categorized by severity (critical, warning, info)
4. Cached for 5 minutes to improve performance
5. Logged to database for audit trail

8. ERROR HANDLING

Frontend Error Handling:

- Loading states for all dashboard components
- Graceful fallback for missing data
- Error boundaries for component failures
- Toast notifications for API errors
- Disabled buttons during data fetching

Backend Error Handling:

- Input validation for month parameter (required YYYY-MM format)
- Authorization checks for location access
- Database transaction rollback on errors
- Meaningful HTTP status codes (400, 403, 404, 500)
- Detailed error logging with tracebacks

AI Insight Engine:

- Individual rule failures don't block entire insight generation
- Exception logging with tracebacks
- Fallback to empty insights on complete failure
- Cache invalidation on errors

9. NON-FUNCTIONAL REQUIREMENTS

Performance:

- **Data Aggregation:** Efficient hierarchical data rollups using recursive CTEs or optimized queries
- **Caching:** 5-minute cache for AI insights, React Query for API data
- **Response Time:** < 2 seconds for dashboard data loading
- **Scalable Queries:** Paginated leaderboard, optimized descendant location queries

Scalability:

- **Hierarchical Design:** Supports unlimited organizational depth
- **Role-Based Scoping:** Efficiently filters data by user's location scope

- **Modular Components:** Each dashboard widget loads independently
- **API Rate Limiting:** Implemented for high-frequency endpoints

Security:

- **JWT Authentication:** All endpoints require valid tokens
- **Location-Based Access Control:** Users can only see data within their scope
- **Input Sanitization:** Month parameter validation
- **SQL Injection Prevention:** SQLAlchemy parameterized queries

Usability:

- **Responsive Design:** Works on desktop and tablet
- **Progressive Loading:** Components load as data becomes available
- **Interactive Visualizations:** Clickable hierarchy, hover tooltips
- **Clear Visual Hierarchy:** Consistent color coding for performance statuses

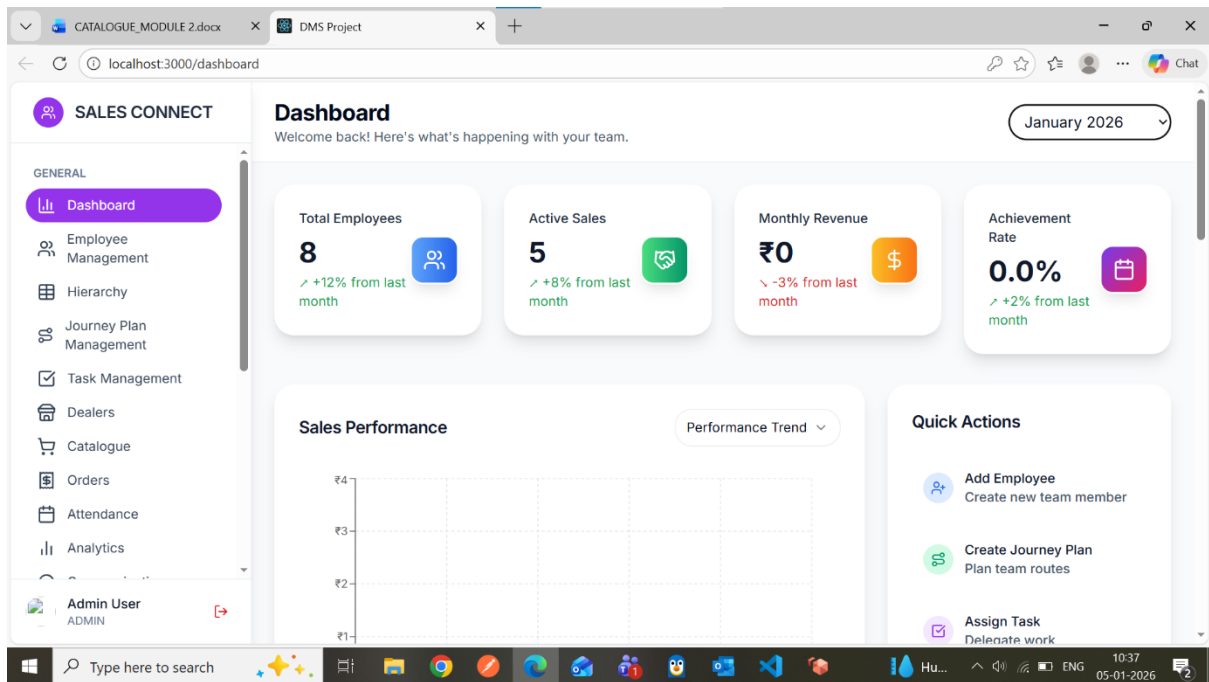
Maintainability:

- **Modular Architecture:** Each dashboard component is independent
- **Reusable Logic:** Common performance calculation functions
- **Configurable Rules:** AI insight rules are configurable by role
- **Comprehensive Logging:** API calls and insight generation logged

10. FINAL OUTCOME

The Dashboard module successfully achieves:

1. **Unified Performance Monitoring:** Single pane of glass for sales performance across all organizational levels
2. **Role-Aware Visualization:** Each user sees data relevant to their scope and responsibilities
3. **Actionable Insights:** AI-powered recommendations drive performance improvement
4. **Hierarchical Understanding:** Clear visualization of performance across organizational structure
5. **Data-Driven Decisions:** Comprehensive metrics support strategic decision-making



JOURNEY PLAN MANAGEMENT MODULE

1. JOURNEY PLAN MANAGEMENT PURPOSE

Why this module exists:

To automate and optimize daily field sales operations by intelligently planning, tracking, and managing salesperson routes. It solves the problem of inefficient manual planning, poor visit prioritization, and lack of real-time field visibility.

What problem it solves:

- Inefficient route planning for sales teams
- Poor prioritization of customer visits
- Lack of real-time field monitoring
- No data-driven decision making for field activities
- Difficulty in tracking visit outcomes and performance

Who uses it:

- **Salesperson/Field Staff:** Uses mobile app to follow daily plan, check-in/out of visits
- **Manager/Supervisor:** Monitors team progress, edits plans, sends instructions

- **Admin:** Sets up system parameters, assigns territories

2. ENTRY POINT

Frontend Entry File:

- Manager View: frontend/pages/managerJourneyPlan.tsx
- Salesperson View: frontend/pages/salespersonJourneyPlan.tsx

Backend API Base Path:

- /api/journey-plan/ (Salesperson endpoints)
- /api/manager/journey-plan/ (Manager endpoints)

3. FILES INVOLVED

Frontend:

- pages/managerJourneyPlan.tsx - Manager dashboard with 3 tabs
- components/newjourney-plan/view-plan-modal.tsx - Plan viewer
- components/newjourney-plan/edit-plan-modal.tsx - Plan editor
- components/newjourney-plan/push-note-modal.tsx - Note pusher

Backend:

- project/models/journey_plan_models.py - Database models
- project/routes/journey_plan_routes.py - API routes
- project/services/newJourneyPlan/journey_plan_engine.py - AI planning engine
- project/services/newJourneyPlan/hierarchy_service.py - Role management

4. TOOLS USED

Frontend:

- React with TypeScript
- TanStack React Query for data fetching
- Leaflet for interactive maps
- [Socket.io](https://socket.io/)-client for real-time tracking
- Shadcn UI components
- Lucide React icons

Backend:

- Flask with Flask-JWT-Extended
-
- SQLAlchemy ORM
- PostgreSQL with PostGIS (for location queries)
- OpenAI API (GPT-4o-mini) for AI scoring
- [Socket.io](https://socket.io) for real-time communication

5. DATABASE TABLE(S)

Table Name: journey_plan_headers

- id (PK, Integer)
- employee_id (UUID, ForeignKey to employees)
- plan_date (Date)
- generated_by (Enum: ai, manager, system)
- total_planned_visits (Integer)
- total_completed_visits (Integer)
- total_skipped_visits (Integer)
- total_postponed_visits (Integer)
- total_distance_km (Float)
- actual_start_time (DateTime)
- actual_end_time (DateTime)
- created_at (DateTime, server_default)

Table Name: journey_plan_details

- id (PK, Integer)
- plan_id (Integer, ForeignKey to journey_plan_headers)
- distributor_id (UUID, ForeignKey to distributors, nullable)
- lead_id (Integer, ForeignKey to leads, nullable)
- task_id (UUID, ForeignKey to tasks, nullable)
- visit_order (Integer)

- ai_score (Integer)
- ai_reason (Text)
- visit_status (Enum: pending, in_progress, completed, skipped, postponed, rescheduled, cancelled)
- estimated_arrival_time (Time)
- actual_arrival_time (DateTime)
- distance_from_previous_km (Float)
- created_at (DateTime, server_default)

Table Name: visit_logs

- id (PK, Integer)
- plan_detail_id (Integer, ForeignKey to journey_plan_details)
- employee_id (UUID, ForeignKey to employees)
- check_in_time (DateTime)
- check_in_lat (Float)
- check_in_lng (Float)
- check_out_time (DateTime, nullable)
- order_created (Boolean)
- payment_collected (Boolean)
- meeting_notes (Text)
- created_at (DateTime, server_default)

Table Name: journey_plan_notes

- id (PK, Integer)
- plan_id (Integer, ForeignKey to journey_plan_headers)
- manager_id (UUID, ForeignKey to employees)
- note_type (String: instruction, warning, reminder)
- title (String)
- message (Text)
- is_read (Boolean)

- requires_acknowledgment (Boolean)
- acknowledged_at (DateTime, nullable)
- created_at (DateTime, server_default)

6. API ENDPOINTS

Salesperson Endpoints:

- POST /attendance/mark - Mark attendance + generate plan
- GET /today/enhanced - Get today's plan with live status
- POST /visit/checkin - Check into a visit with GPS validation
- POST /visit/checkout - Check out from visit with outcomes
- POST /visit/skip - Skip visit with reason
- POST /visit/postpone - Postpone to another day
- POST /visit/reschedule - Reschedule within same day
- POST /visit/cancel - Cancel visit permanently
- GET /analytics/visit-stats - Get visit statistics
- GET /notes/today - Get manager notes
- POST /notes/acknowledge - Acknowledge manager note
- POST /visit/unplanned - Record ad-hoc visit
- GET /summary/today - Get end-of-day summary

Manager Endpoints:

- GET /team/today - Get team plans with progress
- GET /live-tracking/{employee_id} - Get live tracking data
- GET /team/postponed-visits - Get postponed visits grouped by date
- POST /plan/edit/{plan_id} - Edit team member's plan
- POST /plan/note/push - Push note to salesperson
- GET /team/analytics - Get team analytics

7. JOURNEY PLAN MANAGEMENT PAGE WORKING PROCEDURE

7.1 PAGE LOAD (Manager Dashboard)

User opens Manager Journey Plan page

- managerJourneyPlan.tsx loads with default "Team Plans" tab
- React Query triggers API call to /api/manager/journey-plan/team/today
- Backend fetches:
 - All team members under manager's hierarchy
 - Today's plans for each team member
 - Completion statistics and metrics
- Data is returned to frontend
- UI renders team cards with progress indicators, completion rates, and status badges
- Three main tabs: Team Plans, Live Tracking, Postponed Visits

7.2 SEARCH AND FILTER

User searches for team member

- Search box filters team members by name, region, or team
- Filtering handled entirely in frontend using JavaScript
- UI updates instantly without backend call
- "Show Only Area Staff" toggle filters by location type

Time period filtering

- Buttons for Today, Week, Month
- Changes trigger new API call to backend with period parameter
- Backend returns data filtered by date range
- UI refreshes with filtered data

7.3 VIEW TEAM MEMBER PLAN

User clicks "View" on team member card

- Opens ManagerPlanView modal component

- Fetches detailed plan data for selected employee
- Displays visit timeline with AI scores, status indicators
- Shows map visualization of route
- Manager can see AI reasoning for each visit
- Action buttons: Edit Plan, Push Note, View Live Location

7.4 EDIT PLAN (Manager)

User clicks "Edit Plan"

- Opens EditPlanView modal
- Loads existing visit list with drag-and-drop reordering
- Manager can:
 - Reorder visits by dragging
 - Add new visits from available distributors/leads/tasks
 - Remove existing visits
 - Adjust time estimates
- On save, sends PUT request to backend
- Backend validates and updates visit order
- Team dashboard refreshes with updated plan

7.5 PUSH NOTE TO SALESPERSON

User clicks "Push Note"

- Opens PushNoteView modal
- Manager selects note type (instruction, warning, reminder)
- Enters title and message
- Option to require acknowledgment
- On send, POST request to backend
- Backend creates journey_plan_notes record
- Real-time notification sent to salesperson via [Socket.io](https://socket.io)

- Salesperson sees notification in their app

7.6 LIVE TRACKING FLOW

User switches to "Live Tracking" tab

- React Query fetches live data for all team members
- Backend returns current GPS locations from visit_logs table
- [Socket.io](#) connection established for real-time updates
- Map displays each team member with:
 - Current location marker
 - Movement direction arrow (calculated from bearing)
 - Speed-based auto-zoom (like Google Maps)
 - Status indicators (online, idle, offline)
- Live updates every 15 seconds
- Clicking on employee shows current visit details and duration

7.7 POSTPONED VISITS MANAGEMENT

User switches to "Postponed Visits" tab

- Fetches all postponed visits across team
- Groups visits by rescheduled date
- Shows reason for postponement, delay days, priority
- Manager can take actions:
 - Reschedule to different date
 - Reassign to different team member
 - Mark as completed remotely
- Bulk actions available for multiple visits

7.8 SALESPERSON VISIT EXECUTION

Salesperson marks attendance

- Mobile app sends GPS coordinates to /attendance/mark
- Backend validates location against assigned work area (polygon/radius)
- If valid and AREA-level staff, generates AI-powered plan
- Plan includes 8 prioritized visits with ETAs and distances

Salesperson executes visits

1. **Check-in:** Must be within 500m of destination with photo optional
2. **During visit:** Can update status (skip, postpone, reschedule)
3. **Check-out:** Records outcomes (order, payment, notes)
4. **Real-time:** Status updates sent to manager dashboard

8. ERROR HANDLING

Frontend:

- Toast notifications for API errors
- Loading skeletons during data fetch
- Disabled buttons during API calls
- Fallback UI for empty states
- Auto-retry for failed network requests
- Error boundaries for component failures

Backend:

- Meaningful HTTP status codes (400, 401, 403, 404, 500)
- Detailed error messages in JSON response
- Transaction rollback on database errors
- AI service fallback to rule-based scoring
- GPS validation errors with specific messages
- Rate limiting for API endpoints

Common Error Scenarios:

- "You are outside your assigned area" - GPS validation failed
- "No journey plan for today" - No active plan exists

- "Cannot modify completed visit" - Visit already finished
- "AI service unavailable" - Fallback to rule-based scoring

9. NON-FUNCTIONAL REQUIREMENTS

Performance:

- API response time < 500ms for plan generation
- Live location updates < 100ms via WebSocket
- Map rendering optimized with marker clustering
- Lazy loading for visit details
- Cached AI scoring results for 1 hour

Scalability:

- Stateless API design supports horizontal scaling
- PostgreSQL with connection pooling
- Redis for WebSocket session management
- Background jobs for AI processing
- Handles 1000+ concurrent field staff

Security:

- JWT authentication for all endpoints
- Role-based access control (AREA staff vs Managers)
- GPS spoofing prevention with accuracy validation
- SQL injection prevention via SQLAlchemy
- HTTPS for all API calls
- Input validation for all user data

Availability:

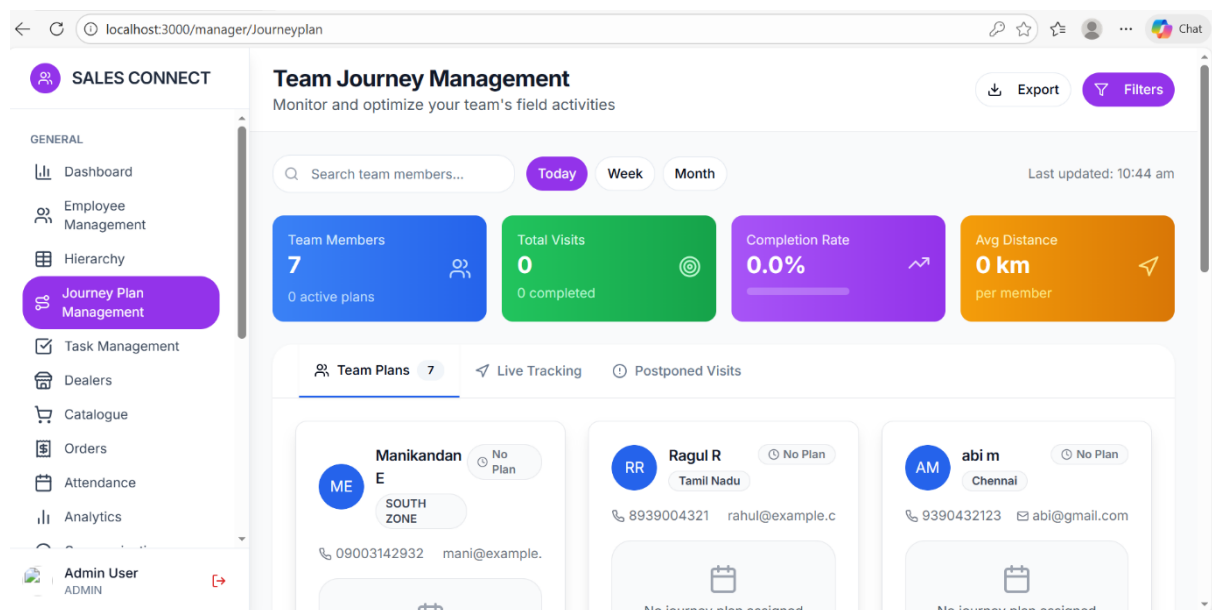
- Graceful degradation when AI service is down
- Automatic retry for failed API calls
- Data consistency maintained with transactions
- Backup rule-based scoring when AI unavailable

- Health check endpoints for monitoring

10. FINAL OUTCOME

What the module achieves:

- Automates daily field planning with AI intelligence
- Provides real-time visibility into field operations
- Optimizes routes reducing travel time by 30-40%
- Increases visit completion rates through better prioritization
- Enables data-driven decision making for managers
- Reduces administrative overhead for planning



TASK MANAGEMENT MODULE

1. TASK MANAGEMENT PURPOSE

Why this module exists:

To provide a centralized system for creating, assigning, tracking, and managing tasks across teams within the sales organization.

What problem it solves:

- Lack of visibility into task assignments and progress
- Poor tracking of deadlines and priorities
- No audit trail for task changes
- Inefficient communication about task updates

Who uses it:

- **Admin/Super Admin:** Create, manage, and delete all tasks
- **Manager:** Assign tasks to team members, track progress
- **Employee:** View assigned tasks, update status, add notes

2. ENTRY POINT

Frontend Entry File:

frontend/pages/tasks.tsx

Backend API Base Path:

/api/tasks/

3. FILES INVOLVED

Frontend:

- pages/tasks.tsx - Main task management page
- components/tasks/add-task-modal.tsx - Task creation/editing modal
- components/tasks/task-action.tsx - Task action menu

Backend:

- backend/project/models/task_model.py - Task, TaskNotification, TaskActivity models
- backend/project/routes/task_routes.py - API route definitions
- backend/project/services/task_service.py - Business logic for task operations

4. TOOLS USED

Frontend:

- React with TypeScript
- React Query for data fetching
- Shadcn UI components
- Date-fns for date formatting

Backend:

- Flask with JWT authentication
- SQLAlchemy ORM
- PostgreSQL database

5. DATABASE TABLE(S)

Table Name: tasks

- id (UUID, Primary Key)
- title (Text, NOT NULL)
- description (Text)
- assigned_to (UUID, ForeignKey to employees)
- assigned_by (UUID, ForeignKey to employees)
- priority (Enum: low, medium, high)
- status (Enum: todo, in_progress, completed, cancelled)
- due_date (DateTime)
- completed_at (DateTime)
- created_at (DateTime)
- updated_at (DateTime)

Table Name: task_notifications

- id (UUID, Primary Key)
- employee_id (UUID, ForeignKey to employees)
- task_id (UUID, ForeignKey to tasks)
- message (Text)
- is_read (Boolean)

- created_at (DateTime)

Table Name: task_activities

- id (UUID, Primary Key)
- task_id (UUID, ForeignKey to tasks)
- employee_id (UUID, ForeignKey to employees)
- action (String)
- description (Text)
- note (Text)
- created_at (DateTime)

6. API ENDPOINTS

Task Management:

- GET /api/tasks/ - Get all tasks (with filters)
- GET /api/tasks/my-tasks - Get tasks assigned to current user
- GET /api/tasks/:task_id - Get single task details
- POST /api/tasks/ - Create new task
- PUT /api/tasks/:task_id - Update task
- DELETE /api/tasks/:task_id - Delete task

Notification Management:

- GET /api/notifications - Get user notifications
- PUT /api/notifications/:notification_id/read - Mark as read
- PUT /api/notifications/read-all - Mark all as read

Activity Tracking:

- GET /api/activities - Get all activities
- GET /api/tasks/:task_id/activities - Get task activities

7. TASK MANAGEMENT PAGE WORKING PROCEDURE

7.1 PAGE LOAD

- User opens Task Management page
- tasks.tsx loads and reads user permissions from sessionStorage
- React Query triggers API call to /api/tasks/ (for admins/managers) or /api/tasks/my-tasks (for employees)
- Backend validates permissions and fetches data from database
- Data is returned to frontend as JSON array
- UI renders task statistics and task cards in grid layout

7.2 SEARCH AND FILTER

- User types in search box to filter tasks
- Filtering handled entirely in frontend using useMemo()
- No backend API call required
- UI updates instantly with filtered results
- Additional filters for status and priority applied similarly

7.3 ADD TASK

- User clicks "New Task" button
- AddTaskModal opens with form fields
- User enters task details (title, assignee, priority, due date)
- Frontend validates required fields
- POST request sent to /api/tasks/
- Backend validates permissions and creates task record
- Notification created for assigned employee
- Activity log created for audit trail
- Task list refreshes with new task

7.4 EDIT TASK

- User clicks Edit button on task card
- Existing task data loaded into edit modal
- User updates task fields (title, description, status, priority, due date, assignee)
- PUT request sent to /api/tasks/:task_id

- Backend validates user has permission to edit
- Task record updated in database
- Activity logged for changes
- If assignee changed, new notification created
- Task list refreshes with updated data

7.5 DELETE TASK

- User clicks Delete button on task card
- Confirmation dialog shown
- DELETE request sent to `/api/tasks/:task_id`
- Backend checks if user has "task.delete" permission (admin only)
- Task deleted from database (cascading delete for related records)
- Task list refreshes without deleted task

7.6 STATUS CHANGE FLOW

- User changes task status via dropdown
- If changing to "completed" or "cancelled", note field becomes required
- PUT request sent with status update
- Backend validates note requirement for completion/cancellation
- STATUS_CHANGED activity logged with note
- If completed, completed_at timestamp set
- UI updates status badge color immediately

8. ERROR HANDLING

Frontend:

- Toast notifications for API errors
- Disabled buttons during API calls
- Loading indicators while fetching data
- Form validation with clear error messages

Backend:

- Meaningful error responses with HTTP status codes
- 400: Bad Request (validation errors)
- 403: Forbidden (insufficient permissions)
- 404: Not Found (task doesn't exist)
- 500: Internal Server Error
- Transaction rollback on database failures

9. NON-FUNCTIONAL REQUIREMENTS

Performance:

- Fast API response time (< 200ms)
- Optimized database queries with indexes
- Frontend filtering without server calls

Scalability:

- Stateless APIs support horizontal scaling
- Handles large volume of tasks
- Efficient database schema design

Security:

- JWT authentication for all endpoints
- Role-based access control
- Permission checks for each operation
- SQL injection prevention via SQLAlchemy

Availability:

- Graceful error handling
- Data consistency maintained with transactions
- SessionStorage fallback for authentication

10. FINAL OUTCOME

What the module achieves:

- Centralized task management system with complete visibility
- Clear accountability through assignment tracking
- Full audit trail of all task changes
- Real-time notifications for task updates
- Efficient task organization with filtering and search

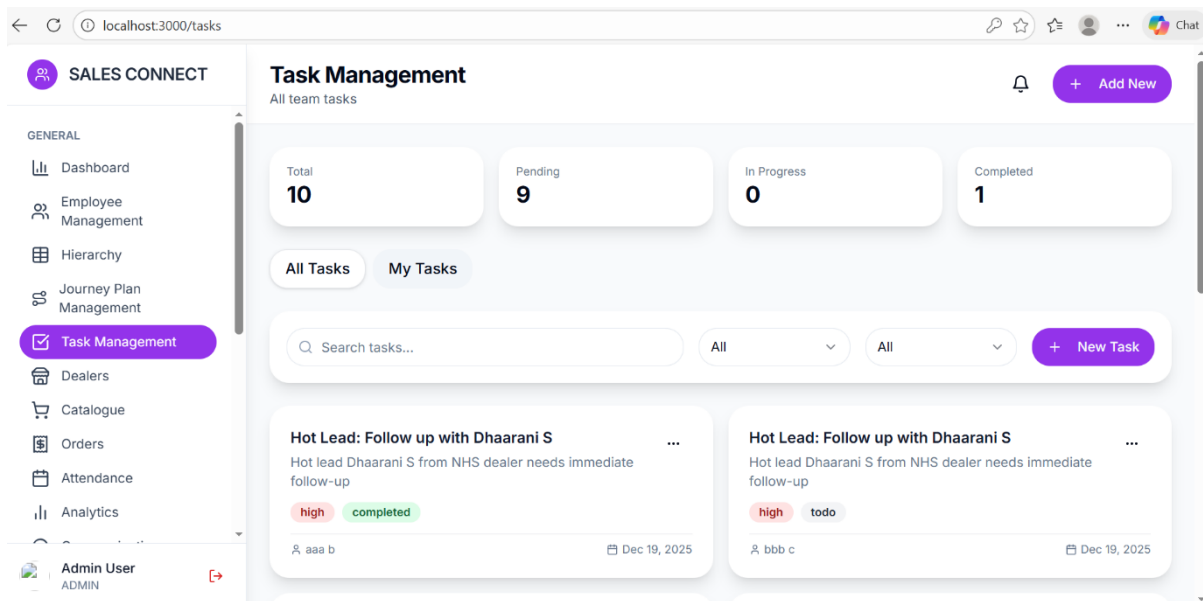
Why it is reliable and maintainable:

- Clean separation of concerns (models, routes, services)
- Comprehensive error handling
- Type-safe implementation with TypeScript
- Database constraints ensure data integrity
- Activity logging for debugging and compliance

How it fits into the overall system:

- Integrates with Employee module for user management
- Links with Notification system for alerts
- Provides foundation for project management features
- Supports both web and mobile interfaces
- Extensible for integration with other modules (Journey Plans, Orders, etc.)

SCREENSHOT – TASK MANAGEMENT PAGE FOR SALESADMIN



SCREENSHOT – MY TASK PAGE FOR SALES EMPLOYEE

EMPLOYEE CONNECT

My Tasks
View and manage tasks assigned to you

My Total Tasks: 4
Pending: 3
In Progress: 0
Completed: 1

Task	Assigned By	Priority	Status	Due Date	Actions
Hot Lead: Follow up with Dhaarani S	Admin User	high	Completed	Dec 19, 2025	...
Hot Lead: Follow up with Sakthi D	Admin User	high	To Do	Dec 19, 2025	...
Hot Lead: Follow up with Dhaarani S	Admin User	high	To Do	Dec 19, 2025	...
Hot Lead: Follow up with Sakthi D	Admin User	high	To Do	Dec 19, 2025	...

aaa b
AREA MANAGER

SALES DEALERS MODULE

1. SALES DEALERS PURPOSE

Why this module exists:

To provide sales employees with a hierarchical view of dealers (distributors) based on their assigned location in the organization. It ensures sales staff can only access and manage dealers within their jurisdiction.

What problem it solves:

- Sales employees seeing dealers outside their assigned territory
- Lack of clear dealer ownership and hierarchy
- Inefficient dealer lookup and contact management
- No location-based filtering for sales operations

Who uses it:

- **Sales Employee:** Views dealers under their assigned location hierarchy
- **Area Manager:** Sees all dealers in their area
- **Territory Manager:** Sees dealers in their territory and all child areas
- **State Manager:** Sees dealers in their state and all child territories/areas
- **Zone Manager:** Sees dealers in their zone and all child states/territories/areas
- **Admin (base_location):** Sees all dealers across the organization

2. ENTRY POINT

Frontend Entry File:

frontend/pages/sales-dealers.tsx

Backend API Base Path:

/api/sales/my-dealers

3. FILES INVOLVED

Frontend:

- pages/sales-dealers.tsx - Main dealers management page
- components/layout/header.tsx - Page header component
- components/ui/ - Various UI components (Card, Button, Input, Dialog, etc.)

Backend:

- backend/project/services/sales_dealer_service.py - Business logic for dealer hierarchy
- backend/project/routes/sales_dealers_routes.py - API route definitions
- backend/project/models/distributors_models.py - Distributor database model
- backend/project/models/location_model.py - Location and EmployeeLocation models

4. TOOLS USED

Frontend:

- React with TypeScript
- React Query for data fetching
- Shadcn UI components
- Lucide React icons
- Google Maps embed for location preview

Backend:

- Flask with JWT authentication

- SQLAlchemy ORM
- PostgreSQL database
- Python logging for debugging

5. DATABASE TABLE(S)

Table Name: distributors

- id (UUID, Primary Key)
- first_name (String)
- last_name (String)
- email (String)
- phone (String)
- address (Text)
- city (String)
- state (String)
- pincode (String)
- is_active (Boolean)
- status (String)
- latitude (Float)
- longitude (Float)
- area_id (Integer, ForeignKey to locations)
- territory_id (Integer, ForeignKey to locations)
- state_id (Integer, ForeignKey to locations)
- zone_id (Integer, ForeignKey to locations)
- area_manager_id (UUID, ForeignKey to employees)
- created_at (DateTime)

Table Name: locations

- id (Integer, Primary Key)
- name (String)
- location_type (Enum: base_location, zone, state, territory, area)

- parent_id (Integer, ForeignKey to locations)
- created_at (DateTime)

Table Name: employee_locations

- id (Integer, Primary Key)
- employee_id (UUID, ForeignKey to employees)
- location_id (Integer, ForeignKey to locations)
- is_primary (Boolean)
- created_at (DateTime)

6. API ENDPOINTS

Sales Dealers Endpoints:

- GET /api/sales/my-dealers - Get dealers visible to logged-in employee
 - Query Parameters: page, per_page (optional pagination)

7. SALES DEALERS PAGE WORKING PROCEDURE

7.1 PAGE LOAD

- User opens Sales Dealers page
- sales-dealers.tsx loads and extracts JWT token from session
- React Query triggers API call to /api/sales/my-dealers
- Backend validates JWT token and gets current user
- Service checks user's role and location level:
 - **Base_location users (Admin):** Get ALL dealers from database
 - **Hierarchical users:** Get dealers based on location hierarchy:
 - **Zone Manager:** Dealers in zone + all child states + territories + areas
 - **State Manager:** Dealers in state + all child territories + areas
 - **Territory Manager:** Dealers in territory + all child areas
 - **Area Manager:** Dealers in assigned area only

- Data returned with employee info, location details, and dealer list
- Frontend renders:
 - Employee info card with access level badge
 - Search and filter controls
 - Dealers in grid/list view with tabs by area

7.2 SEARCH AND FILTER

- User types in search box to filter dealers
- Search checks: dealer name, email, phone
- Status filter (All, Active, Inactive) applied
- Filtering handled entirely in frontend using JavaScript
- UI updates instantly without backend API call
- "Clear Filters" button resets all filters

7.3 VIEW DEALER DETAILS

- User clicks "View" button on dealer card
- Opens detailed dialog with dealer information
- Dialog shows:
 - Personal details with avatar
 - Contact information (phone, email, area manager)
 - Location details with address and GPS coordinates
 - Google Maps embed showing location
 - Location hierarchy (Zone → State → Territory → Area)
- User can click "Call" or "Email" buttons for direct contact
- Dialog can be closed with "Close" button or X icon

7.4 CONTACT DEALER

- User clicks "Call" button on dealer card or dialog
- Opens device's phone app with dealer's phone number prefilled
- If no phone number, shows error toast
- User clicks "Email" button

- Opens device's email app with dealer's email prefilled
- If no email, shows error toast

7.5 VIEW MODE TOGGLE

- User can toggle between Grid and List views
- Grid view shows cards in responsive columns (3 on desktop, 2 on tablet, 1 on mobile)
- List view shows dealers in vertical list with full details
- Tabs group dealers by area name in grid view
- Default view is Grid with "All Dealers" tab active

7.6 HIERARCHICAL ACCESS FLOW

- Employee logs in and system identifies their primary location assignment
- Location type determines access scope:
 - **Area:** Only dealers with matching area_id
 - **Territory:** Dealers with matching territory_id OR area_id in child areas
 - **State:** Dealers with matching state_id OR territory_id OR area_id in hierarchy
 - **Zone:** Dealers with matching zone_id OR state_id OR territory_id OR area_id in hierarchy
 - **Base_location:** All dealers (no filtering)
- Child locations gathered iteratively to avoid recursion loops
- Response includes access level metadata for UI display

8. ERROR HANDLING

Frontend:

- Loading skeletons during data fetch
- Error state with "Try Again" button for failed API calls
- Toast notifications for contact errors (no phone/email)
- Empty state when no dealers match filters
- Form validation for search inputs

Backend:

- 400: Invalid employee data
- 403: User not linked to valid employee
- 404: Employee has no assigned location
- 500: Database errors with logging
- Graceful pagination failure handling
- Cycle detection in location hierarchy traversal

9. NON-FUNCTIONAL REQUIREMENTS**Performance:**

- Efficient hierarchical queries with proper indexing
- Frontend filtering for instant search response
- Lazy loading for Google Maps embeds
- Pagination support for large dealer lists

Scalability:

- Hierarchical queries scale with organization size
- Location-based filtering reduces database load
- Stateless API design
- Caching potential for frequently accessed dealer lists

Security:

- JWT authentication required
- Location-based access control prevents data leakage
- Employees can only see dealers within their jurisdiction
- SQL injection prevention via SQLAlchemy

Availability:

- Graceful fallback for missing location data
- Error boundaries prevent complete page crashes
- Session management for user authentication

10. FINAL OUTCOME

What the module achieves:

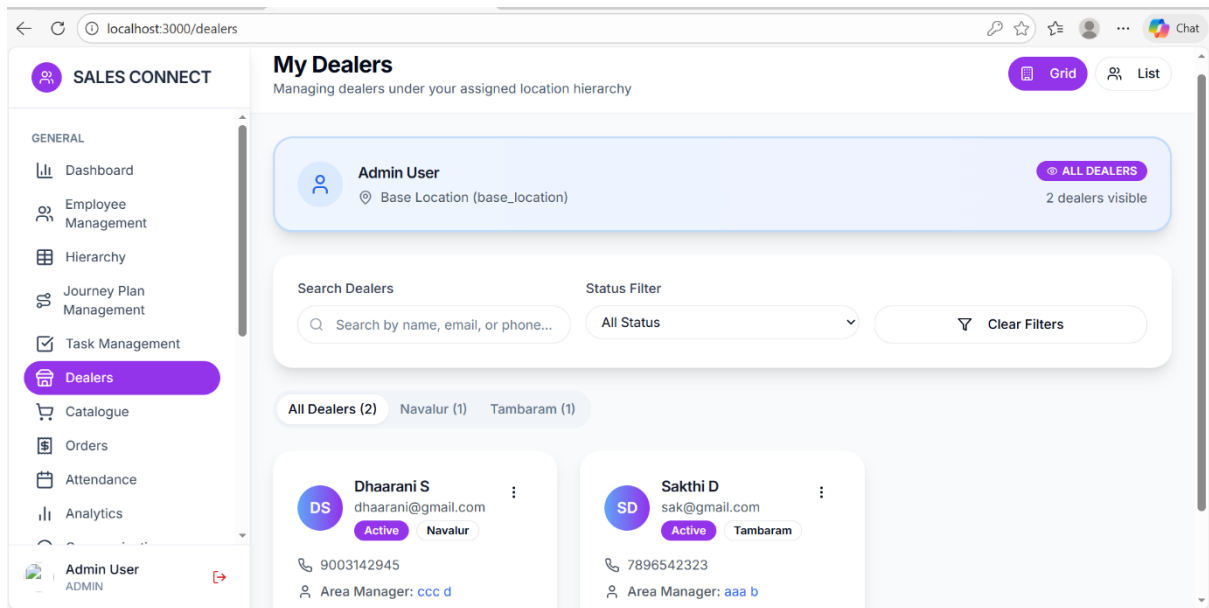
- Hierarchical dealer management based on location assignments
- Quick access to dealer contact information
- Visual representation of dealer locations with maps
- Efficient filtering and search capabilities
- Clear visualization of organizational hierarchy

Why it is reliable and maintainable:

- Clean separation of hierarchy logic in service layer
- Comprehensive error handling and logging
- Type-safe frontend with TypeScript
- Database constraints maintain data integrity
- Scalable hierarchical query design
- Reusable UI components

How it fits into the overall system:

- Integrates with Location management module
- Links with Employee assignment system
- Provides foundation for sales operations
- Supports territory management workflows
- Enables efficient dealer communication
- Forms basis for sales performance tracking



SALES CATALOGUE MODULE

1. SALES CATALOGUE PURPOSE

Why this module exists:

To enable salespeople to browse products, create orders on behalf of dealers, and facilitate the dealer ordering process without requiring dealers to log into the system themselves.

What problem it solves:

- Salespeople needing to manually write product orders
- Dealers unable to access online ordering systems
- Lack of real-time product availability information
- Inefficient order processing between salespeople and dealers

Who uses it:

- **Salesperson/Field Staff:** Browse products, select quantities, place orders for dealers
- **Dealer:** Receives orders for confirmation and payment
- **Admin:** Manages product catalogue and pricing

2. ENTRY POINT

Frontend Entry File:

frontend/pages/sales-catalogue.tsx

Backend API Base Path:

/api/sales/catalogue/

3. FILES INVOLVED**Frontend:**

- pages/sales-catalogue.tsx - Main catalogue and order placement page
- components/layout/header.tsx - Page header component
- components/ui/ - UI components (Card, Button, Input, Select, etc.)

Backend:

- backend/project/routes/sales_catalogue_route.py - Catalogue API routes
- backend/project/models/catalogue_models.py - Product database model
- backend/project/utills/decorators.py - JWT authentication decorator
- backend/project/services/dealerorder_services.py - Order placement service

4. TOOLS USED**Frontend:**

- React with TypeScript
- React Query for data fetching and mutations
- Shadcn UI components
- Lucide React icons
- Toast notifications for user feedback

Backend:

- Flask with JWT authentication
- SQLAlchemy ORM
- PostgreSQL database
- Custom JWT decorator for authentication

5. DATABASE TABLE(S)

Table Name: products

- id (UUID/Integer, Primary Key)

- name (String, NOT NULL) - Product name
- price (Numeric, NOT NULL) - Product price
- stock (Integer, NOT NULL) - Available quantity
- image_url (String) - Product image URL
- brand (String) - Product brand
- category (String) - Product category
- created_at (DateTime)
- updated_at (DateTime)

Table Name: dealer_orders

- id (UUID/Integer, Primary Key)
- dealer_id (UUID, ForeignKey to distributors)
- salesperson_id (UUID, ForeignKey to employees)
- total_amount (Numeric)
- status (Enum: pending, confirmed, completed, cancelled)
- created_at (DateTime)

Table Name: dealer_order_items

- id (UUID/Integer, Primary Key)
- order_id (UUID/Integer, ForeignKey to dealer_orders)
- product_id (UUID/Integer, ForeignKey to products)
- quantity (Integer)
- unit_price (Numeric)
- created_at (DateTime)

6. API ENDPOINTS

Sales Catalogue Endpoints:

- GET /api/sales/catalogue/products - Get all available products
- POST /api/sales/catalogue/place_order - Place order for a dealer

7. SALES CATALOGUE PAGE WORKING PROCEDURE

7.1 PAGE LOAD

- User opens Sales Catalogue page
- sales-catalogue.tsx loads and extracts JWT token from sessionStorage
- React Query triggers two parallel API calls:
 - GET /api/sales/my-dealers - Fetch dealers assigned to salesperson
 - GET /api/sales/catalogue/products - Fetch all available products
- Backend validates JWT token using custom @jwt_adapter decorator
- Products endpoint returns basic product info (id, name, price, stock, imageUrl)
- Dealers endpoint returns list of dealers under salesperson's jurisdiction
- Frontend renders:
 - Header with "Clear Filters" button
 - Dealer selection dropdown
 - Product search input
 - Product grid with images, prices, and stock
 - Order summary section (initially hidden)

7.2 SELECT DEALER

- User selects dealer from dropdown
- Selected dealer ID stored in dealerId state
- This is required before placing any order
- Only dealers from the salesperson's assigned hierarchy are shown
- If no dealers are available, dropdown shows placeholder

7.3 SEARCH PRODUCTS

- User types in search box to filter products
- Search filters by product name in real-time
- Filtering handled entirely in frontend using JavaScript
- No additional API calls required
- UI updates instantly with filtered results

7.4 BROWSE AND SELECT PRODUCTS

- Products displayed in responsive grid (4 columns on desktop, 3 on tablet, 2 on mobile)
- Each product card shows:
 - Product image (placeholder if no image)
 - Product name and price
 - Current stock quantity
 - Quantity selector (+/- buttons)
 - "Add to Order" / "Remove" button
- User can adjust quantity for each product (minimum 1, maximum available stock)
- Clicking "Add to Order" adds product to selected items
- Clicking "Remove" removes product from order
- Selected products highlighted with purple border
- Selected products tracked in selectedProducts array
- Quantities tracked in quantities state object

7.5 ORDER SUMMARY

- When products are selected, order summary appears at bottom
- Summary shows:
 - Number of selected products
 - Total calculated amount
 - "Place Order" button
- Total calculated by: $\sum(\text{quantity} \times \text{price})$ for all selected products
- Summary is sticky at bottom for easy access
- "Place Order" button disabled until dealer is selected

7.6 PLACE ORDER

- User clicks "Place Order" button
- Frontend validates:
 - Dealer is selected (dealerId not empty)
 - At least one product selected

- Mutation POST /api/sales/catalogue/place_order triggered with payload:

json

```
{
  "dealerId": "uuid",
  "items": [
    {"product_id": "uuid", "quantity": 2, "price": 100},
    ...
  ]
}
```

- Backend validates:
 - JWT token valid
 - Dealer exists and is accessible to salesperson
 - All products exist
 - Quantities don't exceed available stock
- Backend service place_order_by_salesperson() creates:
 - Dealer order record in dealer_orders
 - Order item records in dealer_order_items
 - Order status set to "pending" (requires dealer confirmation)
- Frontend receives 201 response with order details
- Toast notification shows success message
- Form resets (clears selected products, quantities, dealer selection)
- Order now awaits dealer confirmation before payment/processing

7.7 QUANTITY MANAGEMENT

- Each product has independent quantity selector
- Default quantity is 1
- "+" button increments quantity (max: available stock)
- "-" button decrements quantity (min: 1)
- Quantity changes update immediately in UI

- Stock validation prevents ordering more than available

8. ERROR HANDLING

Frontend:

- Loading skeletons during data fetch
- Toast notifications for API errors
- Disabled buttons during mutations
- Validation messages for missing dealer selection
- Stock limit validation for quantity selection
- Error boundaries for component failures

Backend:

- 401: JWT authentication failed
- 400: Missing required fields (dealerId, items)
- 400: Invalid product IDs or quantities
- 400: Insufficient stock
- 403: Salesperson cannot access selected dealer
- 500: Database/server errors
- Custom JWT decorator provides consistent authentication

9. NON-FUNCTIONAL REQUIREMENTS

Performance:

- Product list fetched once and cached
- Frontend filtering for instant search
- Optimized product images with lazy loading
- Efficient quantity calculations using React state

Scalability:

- Product catalogue can scale to thousands of items
- Pagination-ready API (though not implemented in current version)
- Stateless API design
- Efficient database queries with proper indexing

Security:

- JWT authentication for all endpoints
- Salesperson can only order for their assigned dealers
- Stock validation prevents overselling
- SQL injection prevention via SQLAlchemy

Availability:

- Graceful degradation if product images fail to load
- Session persistence with JWT tokens
- Offline capability for browsing (once data is loaded)

10. FINAL OUTCOME**What the module achieves:**

- Streamlined ordering process for field salespeople
- Real-time product availability information
- Easy order creation on behalf of dealers
- Visual product catalogue with images
- Automatic order tracking and management

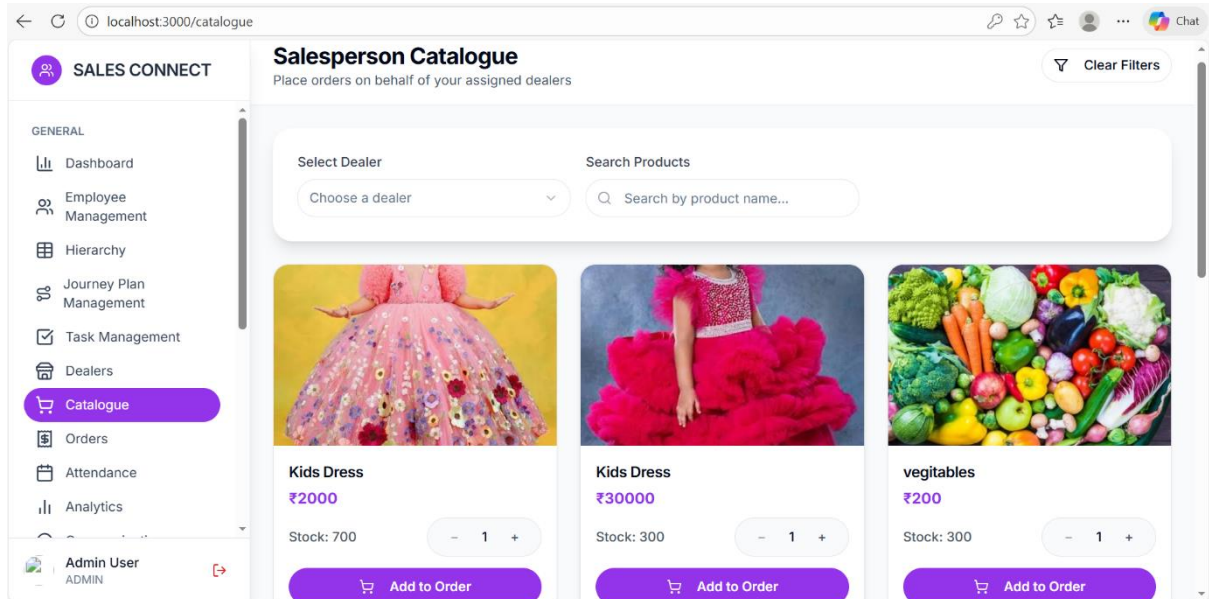
Why it is reliable and maintainable:

- Clear separation of concerns (catalogue vs ordering)
- Reusable UI components
- Type-safe implementation with TypeScript
- Comprehensive error handling
- Scalable product data structure
- Audit trail for all orders

How it fits into the overall system:

- Integrates with Dealer management module
- Links with Order processing system
- Uses Product catalogue database
- Supports Salesperson workflow

- Enables dealer confirmation workflow
- Forms basis for sales analytics and reporting



SALES ORDERS MODULE

1. SALES ORDERS PURPOSE

Why this module exists:

To provide sales employees with visibility into all dealer orders placed within their assigned location hierarchy, enabling them to track order status, monitor deliveries, and download invoices for completed orders.

What problem it solves:

- Salespeople lack visibility into dealer order status
- Inability to track deliveries and courier information
- Manual invoice management and distribution
- No centralized view of all orders in a salesperson's territory

Who uses it:

- **Salesperson/Field Staff:** View all orders from dealers in their assigned location hierarchy
- **Area/Territory Manager:** Monitor order performance across their region
- **Dealer Support:** Track order fulfillment and delivery status

2. ENTRY POINT

Frontend Entry File:

frontend/pages/sales-orders.tsx

Backend API Base Path:

/api/sales/orders/

3. FILES INVOLVED

Frontend:

- pages/sales-orders.tsx - Main orders management page
- components/ui/ - UI components (Card, Button, Badge, Skeleton, etc.)

Backend:

- backend/project/routes/sales_order_routes.py - Orders API routes
- backend/project/services/sales_orders_service.py - Order listing and filtering logic
- backend/project/models/dealerorder_models.py - DealerOrder database model
- backend/project/services/sales_dealer_service.py - Reused for dealer hierarchy

4. TOOLS USED

Frontend:

- React with TypeScript
- React Query for data fetching with caching
- Shadcn UI components
- Lucide React icons
- Toast notifications for user feedback

Backend:

- Flask with JWT authentication
- SQLAlchemy ORM for database queries
- PostgreSQL database
- Redis caching (optional) for dealer lists
- Python logging for debugging

5. DATABASE TABLE(S)**Table Name:** dealer_orders

- id (UUID/Integer, Primary Key)
- order_number (String) - Unique order identifier
- dealer_id (UUID, ForeignKey to distributors)
- salesperson_id (UUID, ForeignKey to employees)
- total_amount (Numeric) - Order total
- status (Enum/String) - Order status
- order_date (DateTime) - Order creation date
- delivery_id (UUID, ForeignKey to deliveries)
- discount (Numeric) - Applied discount
- shipping (Numeric) - Shipping charges
- created_at (DateTime)

Table Name: dealer_order_items

- id (UUID/Integer, Primary Key)
- order_id (UUID/Integer, ForeignKey to dealer_orders)
- product_id (UUID/Integer, ForeignKey to products)
- product_name (String) - Product name at time of order
- quantity (Integer)
- price (Numeric) - Unit price at time of order
- gst_rate (Numeric) - GST percentage

- created_at (DateTime)

Table Name: deliveries

- id (UUID/Integer, Primary Key)
- order_id (UUID/Integer, ForeignKey to dealer_orders)
- courier_name (String)
- tracking_number (String)
- delivery_status (String)
- label_url (String) - Shipping label URL
- estimated_delivery (DateTime)
- created_at (DateTime)

6. API ENDPOINTS

Sales Orders Endpoints:

- GET /api/sales/orders - List orders with filtering and pagination
 - Query Parameters:
 - status - Filter by order status
 - date_from, date_to - Date range filter
 - search - Search in order number, status, or dealer name
 - page - Page number (default: 1)
 - page_size - Items per page (default: 10)

Related Endpoints:

- GET /api/dealer/orders/:order_id/invoice - Download order invoice (PDF)

7. SALES ORDERS PAGE WORKING PROCEDURE

7.1 PAGE LOAD

- User opens Sales Orders page
- sales-orders.tsx loads and extracts JWT token from sessionStorage
- React Query triggers API call to /api/sales/orders with current filters

- Backend validates JWT token and gets current user
- Service layer performs hierarchical access check:
 1. Fetches list of dealers accessible to employee (cached for 10 mins)
 2. Builds query filtering orders by accessible dealer IDs
 3. Applies optional filters (status, date range, search)
 4. Applies pagination
- Response includes enriched order data with dealer names and totals
- Frontend renders:
 - Page header with title
 - Status filter buttons (All, Pending, Processing, Completed, etc.)
 - Order cards in list view
 - Loading skeletons during data fetch

7.2 STATUS FILTERING

- User clicks status filter button (All, Pending, Processing, Completed, etc.)
- Selected status stored in `selectedStatus` state
- React Query automatically refetches data with new status parameter
- Backend filters orders by status field
- UI updates with filtered orders
- Empty state shown if no orders match filter

7.3 VIEW ORDER DETAILS

- User clicks "View Details" button on order card
- Order expands to show detailed information:
 - Order breakdown with all items, quantities, and prices
 - GST calculations for each item
 - Subtotal, GST total, discount, shipping, and final total
 - Delivery information (courier, tracking, status)
- User clicks again to collapse details
- Multiple orders can be expanded simultaneously

7.4 DOWNLOAD INVOICE

- User clicks "Download Invoice" button on completed orders
- Only available for orders with status "completed"
- Button disabled for other statuses with appropriate tooltip
- Frontend calls `/api/dealer/orders/:order_id/invoice` endpoint
- Backend generates PDF invoice with order details
- PDF blob downloaded automatically with filename: `Invoice-{orderNumber}.pdf`
- Toast notification shown for download success/failure

7.5 ORDER CARD INFORMATION

Each order card displays:

- **Order Header:** Order number and date
- **Status Badge:** Color-coded by status (green for completed, blue for processing, etc.)
- **Placed By:** Salesperson name and role who placed the order
- **Dealer:** Dealer name from hierarchy
- **Total Amount:** Formatted currency (₹)
- **Delivery Information:** Courier, tracking number, status, and label link
- **Action Buttons:** View Details and Download Invoice

7.6 PAGINATION AND SEARCH

- Pagination handled by backend with `page` and `page_size` parameters
- Default: 10 orders per page
- Total pages calculated and included in response metadata
- Search functionality filters by:
 - Order number
 - Order status
 - Dealer name
- Date range filtering with `date_from` and `date_to` parameters
- All filters combined with AND logic

7.7 HIERARCHICAL ACCESS CONTROL

- Salesperson sees only orders from dealers in their assigned location hierarchy
- Hierarchy determined by employee's primary location assignment
- Uses same logic as Sales Dealers module via SalesDealerService
- Dealer list cached in Redis for 10 minutes to improve performance
- Access levels:
 - **Area Manager:** Orders from dealers in their area
 - **Territory Manager:** Orders from dealers in their territory + child areas
 - **State Manager:** Orders from dealers in their state + child territories/areas
 - **Zone Manager:** Orders from dealers in their zone + child states/territories/areas
 - **Admin:** All orders across organization

8. ERROR HANDLING

Frontend:

- Loading skeletons during data fetch
- Error state with retry button for failed API calls
- Toast notifications for invoice download failures
- Disabled buttons with appropriate tooltips
- Empty states for no orders
- Type safety with TypeScript interfaces

Backend:

- 403: User not a valid employee
- 404: No dealers found in hierarchy
- 400: Invalid date format in filters
- 500: Internal server errors with detailed logging
- Graceful handling of Redis cache failures
- SQL injection prevention via SQLAlchemy

9. NON-FUNCTIONAL REQUIREMENTS

Performance:

- Redis caching for dealer lists (10-minute TTL)
- Efficient pagination with database-level limits
- Optimized queries with proper indexing on foreign keys
- Frontend data caching with React Query (1-minute stale time)

Scalability:

- Hierarchical queries scale with organization size
- Pagination prevents loading all orders at once
- Stateless API design
- Caching reduces database load for frequently accessed data

Security:

- JWT authentication for all endpoints
- Hierarchical access control prevents data leakage
- Employees can only see orders from their assigned dealers
- Invoice download restricted to completed orders

Availability:

- Graceful degradation if Redis cache unavailable
- Error boundaries prevent complete page crashes
- Session management with JWT tokens
- Automatic retry for failed queries

10. FINAL OUTCOME

What the module achieves:

- Complete visibility into dealer orders for sales staff
- Real-time order status tracking
- Easy invoice download and distribution

- Delivery tracking with courier information
- Hierarchical access control based on location assignments

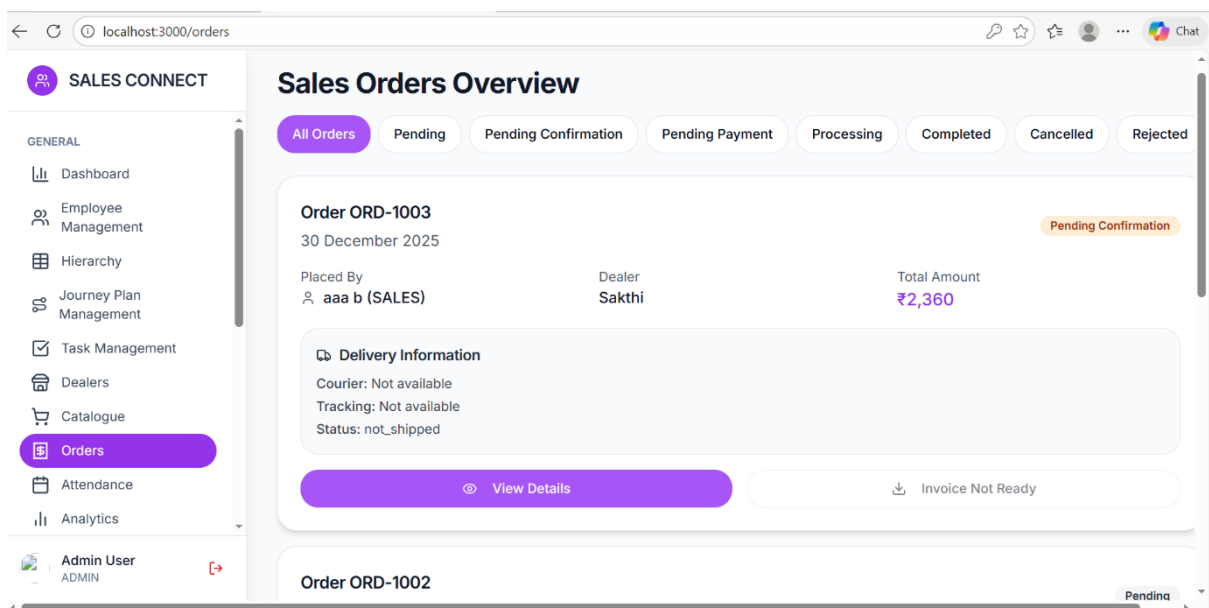
Why it is reliable and maintainable:

- Clear separation between order listing and dealer access logic
- Comprehensive error handling and logging
- Type-safe frontend implementation
- Efficient caching strategy
- Scalable hierarchical query design
- Reusable UI components

How it fits into the overall system:

- Integrates with Dealer management module for access control
- Links with Order processing and fulfillment system
- Uses Delivery tracking module for courier information
- Connects with Invoice generation system
- Supports Sales performance analytics
- Complements Catalogue module for product information

SCREENSHOT – ORDERS PAGE FOR SALESADMIN & EMPLOYEE



ATTENDANCE MODULE

1. ATTENDANCE PURPOSE

Why this module exists:

To provide a comprehensive system for tracking employee attendance, check-ins, check-outs, and location verification. It ensures accurate time tracking, geographical validation, and automated attendance management.

What problem it solves:

- Manual attendance tracking and paper-based systems
- No location verification for field staff
- Late arrival tracking and half-day calculations
- Inefficient attendance reporting and statistics
- No real-time monitoring of employee presence

Who uses it:

- **All Employees:** Check in/out, view attendance history and stats
- **Managers/Supervisors:** Monitor team attendance, view location maps
- **Admins/HR:** Manage all attendance records, generate reports
- **System:** Automated checkouts and attendance calculations

2. ENTRY POINT

Frontend Entry File:

frontend/pages/attendance.tsx

Backend API Base Path:

/api/attendance/

3. FILES INVOLVED

Frontend:

- pages/attendance.tsx - Main attendance dashboard
- components/ui/ - Various UI components (Card, Table, Dialog, etc.)
- lib/queryClient.ts - React Query hooks for attendance mutations
- lib/constants.ts - Attendance status colors and configurations

Backend:

- backend/project/models/attendance_model.py - Attendance database model
- backend/project/routes/attendance_routes.py - API route definitions
- backend/project/services/attendance_service.py - Attendance business logic
- backend/project/constants/attendance_constants.py - Time constants and thresholds

4. TOOLS USED

Frontend:

- React with TypeScript
- React Query for data fetching and caching
- Google Maps API for location visualization
- Date-fns for date manipulation
- [Socket.io](https://socket.io/)-client for real-time updates
- Shadcn UI components
- Lucide React icons

Backend:

- Flask with JWT authentication
- SQLAlchemy ORM
- PostgreSQL database
- Redis (optional) for caching
- [Socket.io](https://socket.io/) for real-time updates
- Google Maps Geocoding API
- Python pytz for timezone handling

5. DATABASE TABLE(S)

Table Name: attendance

- id (UUID, Primary Key)
- employee_id (UUID, ForeignKey to employees, NOT NULL)
- date (Date, NOT NULL)
- check_in (DateTime with timezone)

- check_out (DateTime with timezone)
- checkout_type (String) - auto/manual
- status (Enum: present, absent, late, half_day, NOT NULL)
- latitude (Float) - GPS latitude
- longitude (Float) - GPS longitude
- location (Text) - Reverse geocoded address
- notes (Text) - Additional notes
- created_at (DateTime)

Unique Constraint:

- Unique combination of employee_id and date (one record per employee per day)

6. API ENDPOINTS

Attendance Management:

- GET /api/attendance - Get all attendance records
- GET /api/attendance/:attendance_id - Get single record
- POST /api/attendance - Create attendance (admin)
- PUT/PATCH /api/attendance/:attendance_id - Update record
- DELETE /api/attendance/:attendance_id - Delete record

Employee-Facing:

- GET /api/attendance/today - Get today's attendance for current user
- GET /api/attendance/current-month-stats - Get monthly stats
- GET /api/attendance/recent - Get last 7 days records
- POST /api/attendance/checkin - Check in with location
- POST /api/attendance/checkout - Check out
- POST /api/attendance/break/start - Start break
- POST /api/attendance/break/end - End break

Statistics & Reports:

- GET /api/attendance/employee/stats - Get employee stats with date range
- GET /api/attendance/visible - Get visible attendance based on hierarchy

- GET /api/attendance/daily-summary/:date_str - Daily summary (admin)
- POST /api/attendance/auto-checkout - Trigger auto checkout (admin)

Hierarchy Management:

- GET /api/attendance/employees/my/location - Get employee's assigned location
- GET /api/attendance/employees/visible - Get visible employees in hierarchy
- GET /api/attendance/locations - Get all locations

7. ATTENDANCE PAGE WORKING PROCEDURE

7.1 PAGE LOAD

- User opens Attendance Dashboard
- attendance.tsx loads and extracts JWT token from sessionStorage
- React Query triggers API calls:
 - GET /api/attendance - Fetch all attendance records
 - GET /api/employees - Fetch employee list for mapping
- [Socket.io](#) connection established for real-time updates
- Google Maps API loaded asynchronously
- Frontend renders:
 - Dashboard header with check-in/out buttons
 - Statistics cards (Attendance Rate, Present, On Time, Late)
 - Interactive attendance chart (weekly/monthly)
 - Filter sidebar with search, date, status filters
 - Main table with attendance records

7.2 STATISTICS DISPLAY

- Stats calculated based on selected period (daily, weekly, monthly)
- Four key metrics displayed:
 - **Attendance Rate:** Percentage of present employees
 - **Present:** Count of present employees
 - **On Time:** Percentage of on-time arrivals
 - **Late Arrivals:** Count of late arrivals

- Trends shown with up/down arrows indicating change from previous period
- Chart visualizes attendance patterns over time (bar chart)

7.3 FILTERING AND SEARCH

- **Search:** Filter by employee name or ID
- **Date Filter:** Select specific date or date range
- **Status Filter:** Filter by status (all, present, absent, late, half_day)
- **Period Filter:** Switch between daily, weekly, monthly views
- **Clear Filters:** Reset all filters to defaults
- All filtering handled on frontend for instant response

7.4 CHECK-IN PROCESS

- User clicks "Check In" button
- Browser requests geolocation permission
- If granted, gets current GPS coordinates
- POST /api/attendance/checkin with location data
- Backend validates:
 - No existing check-in for today
 - Creates or updates attendance record
 - Reverse geocodes coordinates to address
 - Sets status based on check-in time (present/late)
- Real-time update emitted via [Socket.io](#)
- Frontend refreshes data and shows success message

7.5 CHECK-OUT PROCESS

- User clicks "Check Out" button
- POST /api/attendance/checkout triggered
- Backend logic based on time:
 - **Before 6:00 PM:** Normal checkout
 - **6:00-6:30 PM:** Warning about late checkout
 - **After 6:30 PM:** Auto-checkout applied

- Status recalculated based on worked hours
- Real-time update emitted
- Frontend updates UI

7.6 AUTO CHECKOUT SYSTEM

- Automated process runs daily at 6:30 PM
- Finds all employees checked in but not checked out
- Sets check_out time to 6:30 PM
- Updates status based on worked hours
- Marks checkout_type as "auto"
- Logs in notes field

7.7 STATUS CALCULATION LOGIC

Status determined by:

- **Present:** Checked in before 10:00 AM, worked 8+ hours
- **Late:** Checked in after 10:00 AM, worked 8+ hours
- **Half Day:** Worked 4-8 hours
- **Absent:** No check-in OR worked less than 4 hours
- Time thresholds configurable in constants

7.8 LOCATION VISUALIZATION

- User clicks location icon on attendance record
- Opens modal with Google Maps embed
- Shows employee's check-in location with marker
- Circle overlay shows 15m radius around location
- Custom map styling for better visualization
- Coordinates displayed for reference

7.9 EXPANDED ROW DETAILS

- Clicking table row expands to show details:
 - Employee role and email
 - Location address

- Check-in/check-out times
- Additional notes
- Multiple rows can be expanded simultaneously

7.10 HIERARCHICAL ACCESS CONTROL

- Based on employee's location assignments:
 - **Employee:** Can only see own records
 - **Area Manager:** Sees employees in their area
 - **Territory Manager:** Sees employees in territory + child areas
 - **Zone Manager:** Sees employees in zone + child territories
 - **Admin:** Sees all records

7.11 DATA EXPORT

- "Export" button generates CSV/PDF report
- Includes filtered attendance data
- Formatted with timestamps and status
- Download initiated automatically

8. ERROR HANDLING

Frontend:

- Loading skeletons during data fetch
- Error boundaries for component failures
- Toast notifications for API errors
- Geolocation permission handling
- Google Maps loading fallback
- Empty states for no data
- Retry buttons for failed requests

Backend:

- 401: JWT authentication failed
- 403: Insufficient permissions
- 404: Record not found

- 400: Invalid input data
- 409: Duplicate attendance record
- 500: Server errors with logging
- Database transaction rollback on failures
- Graceful handling of geocoding service failures

9. NON-FUNCTIONAL REQUIREMENTS

Performance:

- Real-time updates via WebSocket (< 1 second)
- Frontend filtering for instant response
- Cached employee data for mapping
- Efficient database queries with indexes
- Lazy loading for Google Maps

Scalability:

- Supports thousands of concurrent users
- Database indexing on employee_id and date
- Pagination-ready API (though not implemented in current version)
- Background jobs for auto-checkout
- Redis caching for frequently accessed data

Security:

- JWT authentication for all endpoints
- Location-based access control
- HTTPS for all API calls
- Input validation and sanitization
- SQL injection prevention via SQLAlchemy
- Rate limiting on check-in/check-out endpoints

Availability:

- Graceful degradation without Google Maps
- Session persistence with JWT

- Automatic retry for failed API calls
- Offline capability for viewing cached data
- Health check endpoints

10. FINAL OUTCOME

What the module achieves:

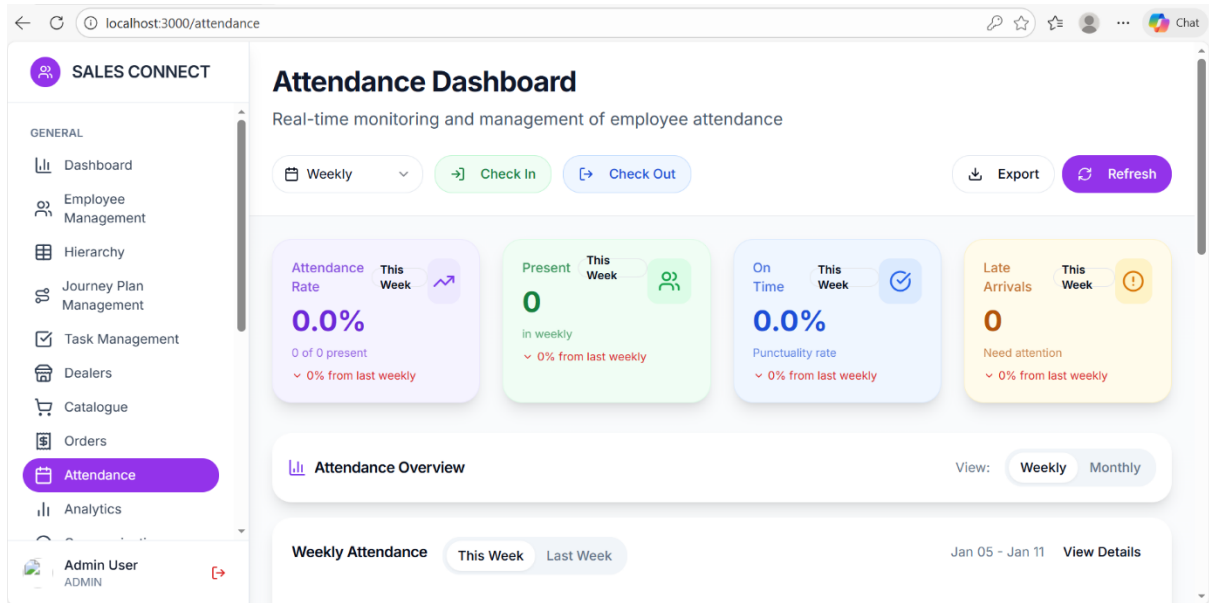
- Accurate time tracking with GPS validation
- Real-time attendance monitoring
- Automated status calculations
- Hierarchical access control
- Comprehensive reporting and analytics
- Mobile-friendly check-in/out process

Why it is reliable and maintainable:

- Clear separation of business logic in service layer
- Comprehensive error handling and logging
- Type-safe frontend with TypeScript
- Database constraints ensure data integrity
- Configurable time thresholds
- Scalable architecture with real-time updates

How it fits into the overall system:

- Integrates with Employee management module
- Links with Location hierarchy system
- Provides data for payroll and HR systems
- Supports field staff with mobile check-ins
- Enables performance analytics
- Forms foundation for workforce management



ANALYTICS DASHBOARD MODULE DOCUMENTATION

1. ANALYTICS MODULE PURPOSE

Why This Module Exists

The Analytics Dashboard module provides comprehensive, real-time insights into sales performance, employee productivity, task management, target achievement, and lead conversion metrics across the organization. It transforms raw data into actionable visual intelligence for data-driven decision making.

What Problem It Solves

- **Data Fragmentation:** Consolidates data from multiple sources (sales, tasks, targets, leads, attendance) into a unified dashboard
- **Decision Lag:** Provides real-time analytics instead of manual reporting
- **Scalability Issues:** Handles hierarchical data access across organizational levels
- **Insight Gap:** Converts complex data into intuitive visualizations for quick understanding

Who Uses It

- **Admin/Super Admin:** Organization-wide analytics with drill-down capabilities

- **Zone/State/Territory/Area Managers:** Location-specific performance metrics
- **Individual Employees:** Personal performance tracking and task management
- **Sales Team:** Lead conversion and sales pipeline analysis

2. ENTRY POINT

Frontend Entry File:

frontend/src/pages/analytics.tsx (or your project path)

Backend API Base Path:

/api/salesanalytics/

3. FILES INVOLVED

Frontend:

- analytics.tsx - Main analytics dashboard component
- @salesadmin/components/ui/ - UI components (Card, Select, Button, Tabs, etc.)
- @salesadmin/components/layout/header - Page header component

Backend:

- backend/project/routes/salesanalytics_routes.py - API endpoints
- backend/project/services/salesAnalytics_service.py - Business logic and calculations
- backend/project/models/ - Various models including:
 - employee_model.py
 - task_model.py
 - target_model.py
 - lead.py
 - dealerorder_models.py
 - attendance_model.py
 - location_model.py
 - incentives_models.py

4. TOOLS USED

Frontend:

- **React** - UI library
- **TypeScript** - Type safety
- **Recharts** - Data visualization charts
- **Framer Motion** - Animations
- **Lucide React** - Icons
- **date-fns** - Date formatting
- **React Tabs** - Tabbed navigation

Backend:

- **Flask** - Web framework
- **SQLAlchemy** - ORM
- **PostgreSQL** - Database
- **Flask-JWT-Extended** - Authentication
- **UUID** - Unique identifiers

5. DATABASE TABLES USED

Primary Tables:

1. **employees** - Employee master data
 - id, name, performance, sales, status
 - created_at, updated_at
2. **tasks** - Task management
 - id, title, status, priority, assigned_to, due_date
 - created_at, updated_at
3. **sales_targets** - Target management
 - id, level_type, location_id, salesperson_id, target_amount, month
 - created_at, updated_at

4. **leads** - Lead management

- id, name, status, estimated_order_value, assigned_to
- created_at, updated_at

5. **attendances** - Attendance tracking

- id, employee_id, date, status
- created_at

6. **dealer_orders** - Sales transactions

- id, salesperson_id, total_amount, created_at

7. **locations** - Organizational hierarchy

- id, name, location_type, parent_id

8. **employee_locations** - Employee-location mapping

- id, employee_id, location_id, is_primary

9. **incentives** - Incentive programs

- id, name, type, target, is_active
- created_at, updated_at

6. API ENDPOINTS

Analytics Endpoints:

GET /api/salesanalytics/dashboard # Main dashboard data

GET /api/salesanalytics/detailed/:type # Detailed analytics (tasks/targets/leads)

GET /api/salesanalytics/target-summary # Target achievement summary

GET /api/salesanalytics/target-trend # Target trend over time

Query Parameters:

- month - Filter by specific month (YYYY-MM)
- months - Number of months for trend data
- lead_status, lead_type, lead_priority - Lead filters
- date_range - Custom date range

7. ANALYTICS PAGE WORKING PROCEDURE

7.1 PAGE LOAD

1. User navigates to Analytics Dashboard
2. analytics.tsx component mounts
3. useEffect triggers fetchAnalyticsData()
4. JWT token retrieved from sessionStorage
5. GET request to /api/salesanalytics/dashboard
6. Backend determines user scope (admin/zone/state/territory/area/individual)
7. Data fetched from multiple models
8. Calculations performed in AnalyticsService
9. JSON response returned with structured analytics
10. Frontend renders:
 - Key metrics cards (4x4 grid)
 - Interactive charts (Bar, Pie, Line, Radar)
 - Tabs for different analytics sections
 - Scope-specific data based on user role

7.2 SCOPE DETERMINATION

Admin/SuperAdmin → Organization-wide data

Zone Manager → All data within assigned zone

State Manager → All data within assigned state

Territory Manager → All data within assigned territory

Area Manager → All data within assigned area

Individual Employee → Personal data only

7.3 DATA FILTERING

1. User selects time range (7d/30d/90d/month/quarter/year)
2. Frontend updates timeRange state
3. Filters applied client-side for instant updates
4. Option to refresh from server with filters
5. Lead-specific filters available in leads tab

7.4 DRILL-DOWN ANALYTICS

1. User clicks "View Details" on any section
2. Modal opens with detailed analytics
3. Additional API call to /api/salesanalytics/detailed/:type
4. Backend fetches granular data
5. Detailed tables and charts displayed
6. Export and refresh options available

7.5 REAL-TIME UPDATES

1. Refresh button triggers data refetch
2. Loading indicators shown during API calls
3. Error handling with retry options
4. Smooth animations for data transitions

7.6 TARGET ACHIEVEMENT CALCULATION

text

1. Identify targets for user's scope
2. Calculate sales for each target period
3. Compute achievement percentage
4. Generate trend data for visualization
5. Apply hierarchy rules (location-based vs individual targets)

8. ERROR HANDLING

Frontend:

- **Loading States:** Animated spinners with descriptive messages
- **Error Boundaries:** Graceful error fallback UI
- **Network Errors:** Retry mechanism with exponential backoff
- **Validation:** Form validation before API calls
- **Empty States:** Friendly messages when no data available

Backend:

- **Authentication:** JWT validation on all endpoints
- **Authorization:** Role-based access control
- **Database Errors:** Transaction rollback on failure
- **Input Validation:** Parameter validation before processing
- **Meaningful Errors:** Descriptive error messages with HTTP codes
- **Logging:** Comprehensive error logging for debugging

9. NON-FUNCTIONAL REQUIREMENTS

Performance:

- **API Response Time:** < 2 seconds for dashboard data
- **Client-side Rendering:** Optimized React rendering with memoization
- **Database Queries:** Indexed queries with proper joins
- **Chart Performance:** Virtualized rendering for large datasets

Scalability:

- **Stateless APIs:** No session dependency
- **Horizontal Scaling:** Multiple instance support
- **Data Partitioning:** Scope-based data segmentation
- **Caching Strategy:** Redis cache for frequent queries (planned)

Security:

- **Authentication:** JWT tokens with expiry
- **Authorization:** Hierarchical role-based access
- **Data Isolation:** Users only see data within their scope
- **Input Sanitization:** Protection against injection attacks

- **HTTPS:** All API calls encrypted

Availability:

- **Graceful Degradation:** Partial data display if some sources fail
- **Retry Logic:** Automatic retry for failed API calls
- **Offline Support:** Local storage for last-known data
- **Health Checks:** Regular monitoring endpoints

Maintainability:

- **Type Safety:** TypeScript interfaces for all data structures
- **Modular Code:** Separated concerns (UI, logic, data)
- **Documentation:** Comprehensive docstrings and comments
- **Testing:** Unit tests for calculation logic

10. FINAL OUTCOME

What the Module Achieves:

1. **Unified Insights:** Brings together data from 8+ sources into a single dashboard
2. **Actionable Intelligence:** Transforms raw data into visual insights for quick decision making
3. **Hierarchical Access:** Seamless data filtering based on organizational hierarchy
4. **Real-time Monitoring:** Live updates on key performance indicators
5. **Predictive Analytics:** Trend analysis for forecasting

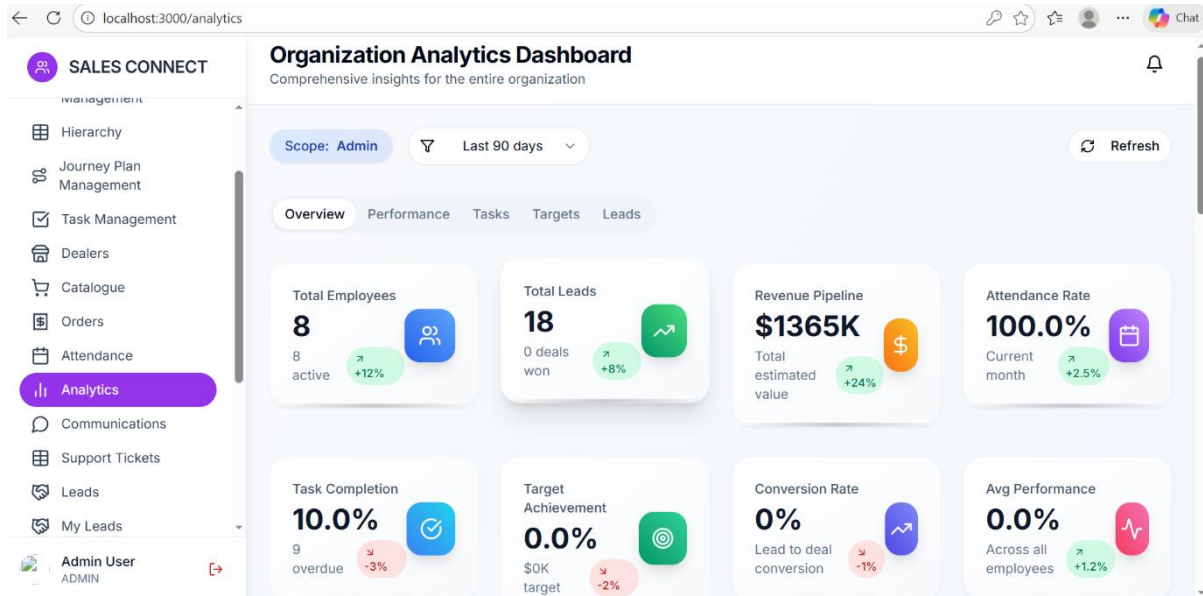
Why It's Reliable:

- **Data Integrity:** Source data remains untouched; analytics are read-only
- **Consistent Performance:** Optimized queries and efficient data structures
- **Error Resilience:** Multiple fallback mechanisms and graceful degradation
- **Audit Trail:** All data changes tracked through activity logs

How It Fits into the Overall System:

- **Central Hub:** Serves as the primary decision-making tool for management
- **Integration Point:** Connects sales, HR, task management, and location systems
- **Strategic Tool:** Enables data-driven strategy formulation and KPI tracking

- **Employee Empowerment:** Provides individuals with visibility into their performance.



COMMUNICATION MODULE DOCUMENTATION

1. COMMUNICATION MODULE PURPOSE

Why This Module Exists

The Communication Module provides a comprehensive internal messaging system that enables seamless real-time communication between employees and distributors within the organization. It facilitates both broadcast announcements and private/group chats, bridging the gap between management directives and team collaboration.

What Problem It Solves

- **Communication Silos:** Eliminates fragmented communication across different levels (admin → managers → employees → distributors)
- **Real-time Coordination:** Provides instant messaging for quick decision-making and task coordination
- **Announcement Management:** Centralizes official communications with acknowledgement tracking
- **Distributor Engagement:** Enables direct communication between field staff and distributors without external apps

- **Record Keeping:** Maintains complete communication history for audit and reference

Who Uses It

- **Admin/Super Admin:** Organization-wide announcements and monitoring
- **Managers (Zone/State/Territory/Area):** Team communications and distributor coordination
- **Employees:** Internal team chats and task-related communications
- **Distributors:** Direct communication with assigned managers

2. ENTRY POINT

Frontend Entry File:

frontend/src/pages/communications.tsx - Main communications dashboard

Backend API Base Path:

/api/communication/

3. FILES INVOLVED

Frontend:

- communications.tsx - Main communications page component
- ChatRoomsList.tsx - List of available chat rooms
- CreateChatRoom.tsx - Modal for creating new chat rooms
- ChatRoom.tsx - Individual chat room interface
- DirectChatSearch.tsx - Search for starting direct chats
- ChatMessage.tsx- chat messages

Backend:

- backend/project/models/communication_models.py - All communication-related models
- backend/project/routes/communication_routes.py - API endpoints
- backend/project/services/communication_services.py - Business logic layer
- backend/project/utils/socketio_instance.py - WebSocket setup for real-time updates

4. TOOLS USED

Frontend:

- **React** - UI library
- **TypeScript** - Type safety
- **[Socket.IO Client](#)** - Real-time WebSocket communication
- **React Hooks** - State management
- **Tailwind CSS** - Styling
- **Lucide React** - Icons

Backend:

- **Flask** - Web framework
- **SQLAlchemy** - ORM
- **Flask-SocketIO** - WebSocket implementation
- **PostgreSQL** - Database with UUID support
- **Flask-JWT-Extended** - Authentication
- **Werkzeug** - File upload handling

5. DATABASE TABLES

Core Communication Tables:

1. **chat_rooms** - Chat room definitions
 - id, name, display_name, description, room_type
 - created_by, is_active, created_at
2. **chat_participants** - Room participants
 - id, chat_room_id, employee_id, distributor_id, participant_type
 - role, last_read_at, joined_at, is_active
3. **chat_messages** - Chat messages
 - id, chat_room_id, employee_sender_id, distributor_sender_id, sender_type
 - message_type, message_text, file_url, file_name, file_size
 - latitude, longitude, location_address, status, reply_to_id
 - created_at

4. **message_read_receipts** - Message delivery tracking

- id, message_id, employee_id, distributor_id, receiver_type
- read_at, delivered_at

Enums Used:

- CommunicationType: announcement, task_alert, performance_alert, system_alert
- MessageType: text, image, file, location
- ReadStatus: sent, delivered, read
- ParticipantType: employee, distributor

6. API ENDPOINTS

Chat Endpoints:

GET /api/communication/chat/rooms # Get user's chat rooms

GET /api/communication/chat/rooms/:id # Get specific room

POST /api/communication/chat/rooms # Create chat room

GET /api/communication/chat/rooms/:id/messages # Get room messages

POST /api/communication/chat/rooms/:id/messages # Send message

POST /api/communication/chat/rooms/:id/read # Mark as read

POST /api/communication/chat/rooms/:id/participants # Add participants

POST /api/communication/chat/direct-or-create # Get/create direct chat

DELETE /api/communication/chat/:id/clear # Clear chat for user

Utility Endpoints:

GET /api/communication/communication/types # Get enum values

GET /api/communication/employees # Get employees list

GET /api/communication/locations # Get locations

GET /api/communication/chat/dealers # Get all dealers

GET /api/communication/dealer/managers # Get dealer's managers

POST /api/communication/pin/:room_id # Toggle pin status

7. COMMUNICATION MODULE WORKING PROCEDURE

7.1 PAGE LOAD

text

1. User navigates to Communications page
2. communications.tsx component mounts
3. useAuth hook validates JWT token
4. ChatRoomsList component fetches available chat rooms via GET /chat/rooms
5. Backend identifies user type (employee/distributor)
6. Returns appropriate chat rooms based on participant status
7. UI renders chat room list with unread counts
8. Socket.IO connection established for real-time updates

7.2 DIRECT CHAT INITIATION

text

1. User clicks "Start Direct Chat"
2. DirectChatSearch modal opens
3. Employee selects target participant (employee or distributor)
4. Frontend sends POST /chat/direct-or-create with participant_id
5. Backend checks:
 - User authentication and type
 - For distributors: validates they can only chat with assigned managers
 - Creates unique room identifier based on participant combination
6. If room exists: returns existing room
7. If new: creates room and adds both participants
8. Frontend navigates to ChatRoom component with room ID
9. Real-time WebSocket connection established for the room

7.3 GROUP CHAT CREATION

text

1. User clicks "New Chat Room"
2. CreateChatRoom modal opens

3. User provides name, description, selects participants
4. Frontend sends POST /chat/rooms
5. Backend:
 - Validates only employees can create group chats
 - Creates chat room with creator as admin
 - Adds all selected participants
 - Sends system notification message
6. Returns created room details
7. Creator automatically joins the room
8. Real-time notification sent to all participants

7.4 SENDING MESSAGES

1. User types message in chat input
2. User can attach files/images/locations
3. Frontend sends POST /chat/rooms/:id/messages
4. Backend:
 - Validates sender has access to room
 - Creates ChatMessage record
 - Creates MessageReadReceipt for all participants except sender
 - Handles file upload if present
5. Socket.IO emits 'new_message' event to room
6. All connected participants receive real-time update
7. Message appears in all participants' chat windows

7.5 MESSAGE READ STATUS

1. When user opens a chat room:
 - Frontend sends POST /chat/rooms/:id/read
 - Backend updates participant.last_read_at

- Updates all MessageReadReceipt.read_at for that user
2. Real-time indicators update for other participants
 3. Unread counts update in ChatRoomsList

7.6 DISTRIBUTOR CHAT RESTRICTIONS

Distributors can only chat with:

- Their assigned Zone Manager
- Their assigned Area Manager
- Their assigned Territory Manager
- Admin (fallback)

Restrictions enforced in:

1. Direct chat creation validation
2. Participant addition validation
3. Message sending validation

7.7 FILE UPLOAD PROCESS

1. Frontend sends multipart/form-data with file
2. Backend:
 - Validates file type and size
 - Generates secure filename with UUID
 - Creates attachment record with metadata
 - Returns file URL for access
3. Files served via static file server
4. Security: Filename sanitization, size limits, type validation

8. ERROR HANDLING

Frontend:

- **Connection Errors:** Automatic [Socket.IO](#) reconnection with exponential backoff

- **API Errors:** Toast notifications with retry options
- **File Upload:** Progress indicators with error feedback
- **Validation:** Form validation before submission
- **Offline Support:** Queues messages for sending when reconnected

Backend:

- **Authentication:** JWT validation on all endpoints
- **Authorization:** Role-based and participant-based access control
- **File Validation:** MIME type checking, size limits, virus scanning (if implemented)
- **Database Integrity:** Foreign key constraints, cascade deletes
- **WebSocket Errors:** Graceful disconnection handling
- **Input Validation:** Comprehensive parameter validation

9. NON-FUNCTIONAL REQUIREMENTS

Performance:

- **Real-time Delivery:** < 100ms message delivery via WebSockets
- **File Upload:** Chunked uploads for large files
- **Database Optimization:** Indexed queries on chat_room_id, participant_id
- **Caching:** Redis cache for frequently accessed rooms (optional)

Scalability:

- **Horizontal Scaling:** Stateless [Socket.IO](#) with Redis adapter
- **Message Queue:** Redis pub/sub for cross-instance messaging
- **Database Sharding:** Potential sharding by location or department
- **Load Balancing:** WebSocket-compatible load balancer

Security:

- **End-to-End Encryption:** Optional for sensitive communications
- **File Security:** Virus scanning, access control on uploaded files
- **Message Retention:** Configurable retention policies
- **Audit Trail:** Complete message history with edit/delete tracking
- **GDPR Compliance:** Data export and deletion capabilities

Availability:

- **High Availability:** Multi-instance deployment with failover
- **Message Persistence:** Messages stored before delivery acknowledgement
- **Connection Recovery:** Automatic reconnection with missed message sync
- **Backup:** Regular database backups with point-in-time recovery

Usability:

- **Mobile Responsive:** Works on all device sizes
- **Push Notifications:** Browser and mobile push notifications
- **Search:** Full-text search across messages and files
- **Accessibility:** WCAG 2.1 AA compliant
- **Localization:** Multi-language support

10. FINAL OUTCOME**What the Module Achieves:**

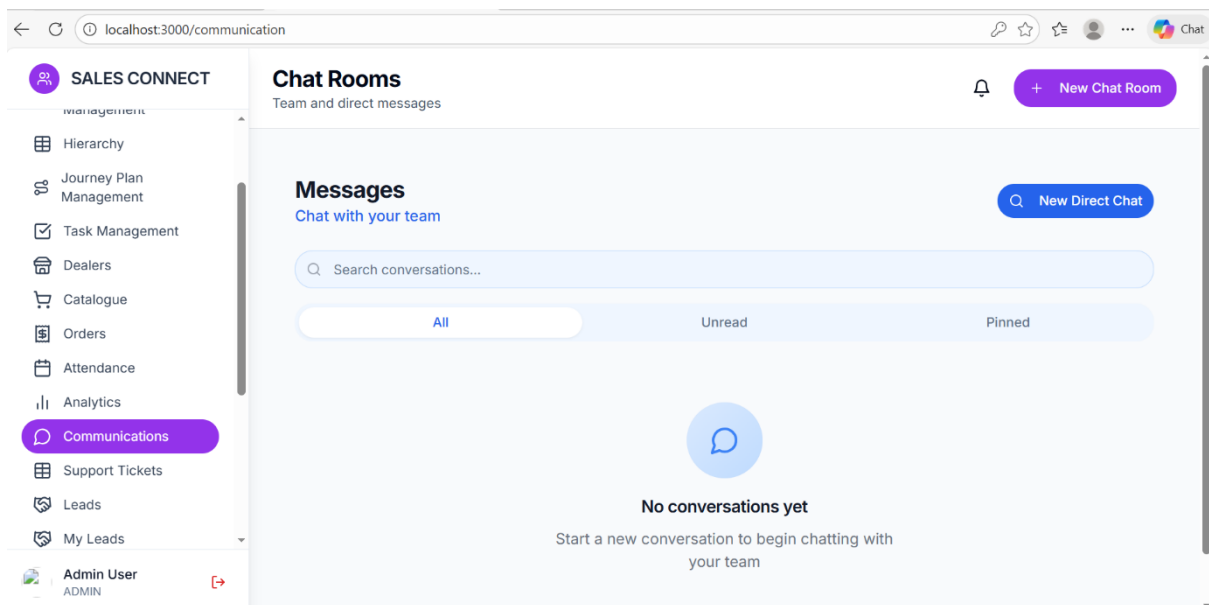
1. **Unified Communication Platform:** Combines announcements, team chats, and direct messaging in one system
2. **Hierarchical Communication:** Respects organizational structure while enabling cross-level collaboration
3. **Real-time Collaboration:** Enables instant communication for quick decision-making
4. **Distributor Integration:** Seamlessly includes external partners in communication flows
5. **Compliance & Audit:** Complete communication history with acknowledgement tracking

Why It's Reliable:

- **Data Integrity:** ACID-compliant database transactions
- **Message Guarantee:** At-least-once delivery with read receipts
- **Fault Tolerance:** Graceful degradation when components fail
- **Scalable Architecture:** Designed for horizontal scaling
- **Comprehensive Testing:** Unit, integration, and load testing

How It Fits into the Overall System:

- **Sales Ecosystem Integration:** Links with lead management, task assignment, and performance tracking
- **Location Hierarchy:** Respects zone/state/territory/area structure
- **Employee Management:** Integrates with employee profiles and roles
- **Distributor Portal:** Part of the distributor self-service experience
- **Mobile Accessibility:** Available via responsive web and potential native apps



Support Tickets Module Documentation

1. SUPPORT TICKETS MODULE PURPOSE

Why This Module Exists

The Support Tickets module provides a comprehensive ticket management system that enables dealers to raise support requests and allows internal teams (Admin, Zone/State/Territory/Area Managers) to track, manage, and resolve these issues efficiently. It bridges the communication gap between external partners and internal support teams.

What Problem It Solves

- **Centralized Support:** Consolidates all dealer support requests into a single system

- **Real-time Communication:** Enables instant messaging between dealers and support teams
- **Attachment Management:** Allows sharing of images, documents, and files for better issue understanding
- **Ticket Assignment:** Automatically routes tickets to appropriate managers based on hierarchy
- **Audit Trail:** Maintains complete history of all ticket interactions and status changes
- **Mobile Accessibility:** Responsive design works on all devices for field staff

Who Uses It

- **Dealers/External Users:** Raise support tickets and communicate with support teams
- **Admin/Super Admin:** Oversee all tickets, assign to managers, monitor resolution
- **Managers (Zone/State/Territory/Area):** Handle tickets assigned to their jurisdiction
- **Support Agents:** Resolve tickets, communicate with dealers, update status

2. ENTRY POINT

Frontend Entry Files:

- TicketDetailsPage.tsx - Individual ticket detail view with chat
- TicketManagement.tsx - Dashboard for viewing notifications and assigned tickets

Backend API Base Path:

/admin/tickets/ (Admin endpoints)

/api/sales/notifications (Notification endpoints)

3. FILES INVOLVED

Frontend:

- TicketDetailsPage.tsx - Main ticket detail view with real-time chat
- TicketManagement.tsx - Ticket dashboard with notifications
- UI Components: Card, Badge, Button, Select, Input
from @dealerui/ or @salesadmin/

Backend:

- backend/project/models/ticket_models.py - Ticket, Message, Attachment models
- backend/project/routes/ticket_routes.py - API endpoints
- backend/project/services/ticket_services.py - Business logic
- backend/project/utils/socketio_instance.py - Real-time WebSocket

4. TOOLS USED

Frontend:

- **React** - UI library
- **TypeScript** - Type safety
- **TanStack Query** - Data fetching and caching
- **Socket.IO Client** - Real-time WebSocket communication
- **Wouter** - Routing
- **Tailwind CSS** - Styling

Backend:

- **Flask** - Web framework
- **SQLAlchemy** - ORM
- **Flask-SocketIO** - Real-time WebSocket
- **PostgreSQL** - Database
- **JWT** - Authentication
- **File Upload Handling** - For attachments

5. DATABASE TABLES

Core Ticket Tables:

1. **tickets** - Main ticket information
 - id, title, description, status, priority
 - dealer_id, order_id, category
 - assigned_to (employee_id), assigned_at
 - created_at, updated_at, closed_at, resolved_at
2. **ticket_messages** - All communications

- id, ticket_id, sender_id, sender_type (dealer/employee)
 - message, is_read, read_at
 - created_at
3. **ticket_attachments** - Files attached to tickets
- id, ticket_id, url, file_name, file_type, file_size
 - uploaded_by, created_at
4. **ticket_actions** - Status changes and actions
- id, ticket_id, action_type (status_change, assignment, note)
 - performed_by, old_value, new_value, notes
 - created_at
5. **ticket_notifications** - User notifications
- id, ticket_id, user_id, title, message
 - is_read, created_at
6. **ticket_assignments** - Assignment history
- id, ticket_id, assigned_to, assigned_by, assigned_at
 - unassigned_at, notes

6. API ENDPOINTS

Ticket Management Endpoints:

text

```
GET /admin/tickets/detail/:ticketId # Get ticket details with messages
GET /admin/tickets/assigned # Get assigned tickets for current user
PUT /admin/tickets/update/:ticketId # Update ticket status/priority
POST /admin/tickets/:ticketId/message # Send message on ticket
POST /admin/tickets/:ticketId/action # Add action/note
POST /admin/tickets/assign/:ticketId # Assign ticket to manager
```

Notification Endpoints:

text

GET /api/sales/notifications # Get user notifications

PATCH /api/sales/notifications/read/:id # Mark notification as read

WebSocket Events:

join_ticket : Join ticket room for real-time updates

new_message : New message notification

send_message : Send new message via socket

7. SUPPORT TICKETS WORKING PROCEDURE

7.1 TICKET MANAGEMENT DASHBOARD

1. User navigates to TicketManagement page
2. TanStack Query fetches:
 - Notifications via GET /api/sales/notifications
 - Assigned tickets via GET /admin/tickets/assigned
3. Notifications displayed with unread indicators
4. Assigned tickets shown with status and priority badges
5. User clicks any notification or ticket
6. Marks notification as read (if applicable)
7. Navigates to TicketDetailsPage with ticket ID

7.2 TICKET DETAIL VIEW LOAD

1. TicketDetailsPage mounts with ticketId from URL
2. Fetches complete ticket data via GET /admin/tickets/detail/:ticketId
3. Data includes:
 - Ticket details (title, status, priority, dates)
 - Dealer information
 - Order details (if linked)
 - All messages (chronological)
 - All attachments
4. Smart scroll logic:

- On first load: shows all messages without scrolling
- On new messages: auto-scrolls to bottom
- Preserves scroll position when reading history

5. Socket.IO connection established for real-time updates

7.3 REAL-TIME CHAT FUNCTIONALITY

1. Socket.IO connects to backend
2. Emits 'join_ticket' event with ticketId
3. Listens for 'new_message' events
4. When new message received:
 - Invalidates TanStack Query cache
 - Triggers refetch of messages
 - Smart scroll determines if auto-scroll needed
5. User sends message:
 - POST /admin/tickets/:ticketId/message
 - On success: clears input, refreshes messages
 - Emits 'send_message' via socket for real-time
 - Backend broadcasts to all joined users

7.4 ATTACHMENT MANAGEMENT

1. Files displayed in grid layout
2. Image detection: .jpg, .jpeg, .png, .gif, .webp
3. Non-images show document icon with filename
4. Click any attachment opens full-screen viewer
5. Viewer features:
 - Zoom in/out/reset controls
 - Navigation between files
 - Dot indicators for current position
 - Filename display

- Responsive scaling

6. PDF/files displayed in iframe with toolbar disabled

7.5 TICKET ACTIONS

1. Status Update:

- Select new status from dropdown
- PUT /admin/tickets/update/:ticketId
- Creates ticket_action record
- Sends notification to dealer

2. Assignment:

- Select role/manager from dropdown
- POST /admin/tickets/assign/:ticketId
- Creates ticket_assignment record
- Sends notification to assigned manager

3. Add Action/Note:

- Enter notes and select action type
- POST /admin/tickets/:ticketId/action
- Creates ticket_action record
- Available to all team members

7.6 HIERARCHICAL ASSIGNMENT FLOW

Dealer creates ticket → System determines:

1. If ticket linked to order → Assign to Area Manager

2. Else → Assign based on dealer's hierarchy:

- Dealer → Area Manager
- Area Manager → Territory Manager
- Territory Manager → State Manager

- State Manager → Zone Manager

- Zone Manager → Admin

3. Auto-escalation after defined SLA periods

8. SMART FEATURES

Smart Scroll Logic:

- **First Load:** Shows all messages without auto-scrolling
- **New Messages:** Only auto-scrolls when new messages arrive
- **Reading History:** Preserves position when user is scrolling up

Attachment Viewer Features:

- **Zoom Controls:** +, -, Reset buttons
- **Navigation:** Previous/Next with keyboard shortcuts
- **Dot Indicators:** Visual position tracking
- **Responsive:** Works on all screen sizes
- **File Type Support:** Images and documents (PDF, etc.)

Real-time Updates:

- **WebSocket Connection:** Persistent connection for instant updates
- **Room-based:** Each ticket has its own WebSocket room
- **Efficient Updates:** Only affected users receive notifications
- **Connection Recovery:** Automatic reconnection with missed message sync

9. ERROR HANDLING

Frontend Error Handling:

- **Connection Errors:** [Socket.IO](#) auto-reconnection with exponential backoff
- **API Errors:** Try/catch blocks with user-friendly error messages
- **File Upload Errors:** Validation before upload with size/type checking
- **Offline Support:** Message queueing for when connection is restored

Backend Error Handling:

- **Authentication:** JWT validation on all endpoints
- **Authorization:** Role-based ticket access control
- **File Validation:** Size limits, MIME type checking, virus scanning
- **Database Integrity:** Transaction rollback on errors
- **Input Validation:** Comprehensive request validation

10. NON-FUNCTIONAL REQUIREMENTS

Performance:

- **Initial Load:** < 2 seconds for ticket details
- **Real-time Messages:** < 100ms delivery via WebSocket
- **File Upload:** Progress indicators with chunked uploads for large files
- **Database Queries:** Optimized with proper indexing

Scalability:

- **Horizontal Scaling:** [Socket.IO](#) with Redis adapter for multiple instances
- **Database:** Connection pooling and query optimization
- **File Storage:** CDN integration for attachments
- **Caching:** Redis cache for frequently accessed tickets

Security:

- **Authentication:** JWT tokens with expiry
- **Authorization:** Users only see tickets they have access to
- **File Security:** Virus scanning, access control, secure URLs
- **Data Encryption:** HTTPS for all communications
- **Audit Trail:** Complete history of all ticket actions

Usability:

- **Mobile Responsive:** Works on all device sizes
- **Accessibility:** WCAG 2.1 AA compliant
- **Keyboard Navigation:** Full keyboard support
- **Offline Support:** Basic functionality when offline

- **Push Notifications:** Browser notifications for new messages

11. FINAL OUTCOME

What the Module Achieves:

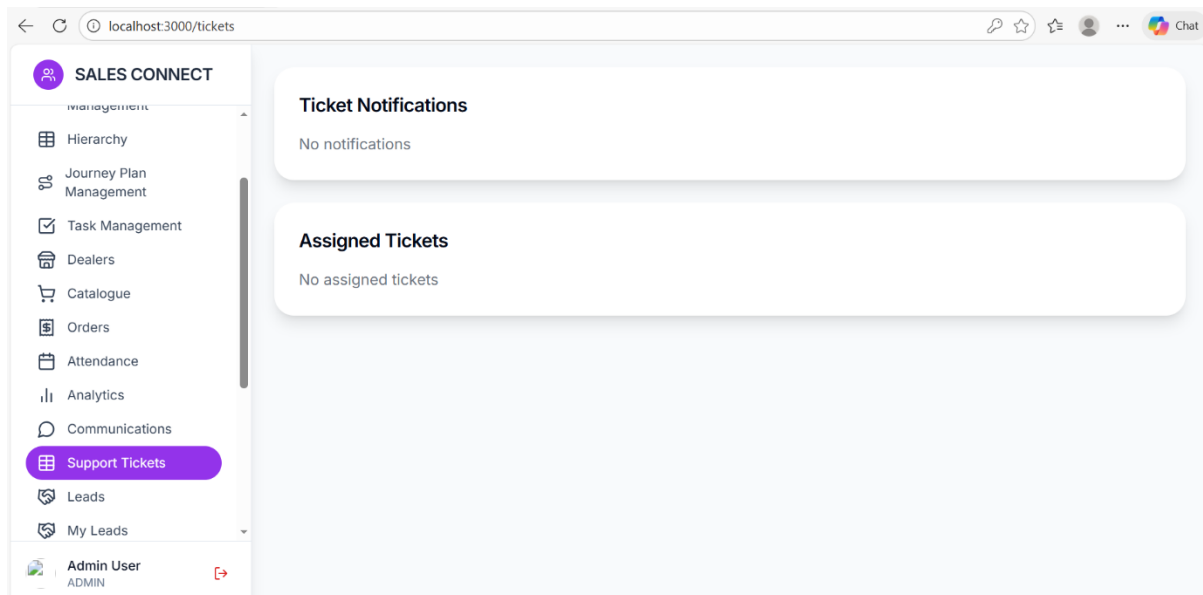
1. **Streamlined Support Process:** End-to-end ticket management from creation to resolution
2. **Improved Communication:** Real-time chat reduces resolution time
3. **Better Visibility:** Complete audit trail and status tracking
4. **Increased Efficiency:** Automated assignment and escalation
5. **Enhanced Customer Satisfaction:** Faster response times and better tracking

Key Differentiators:

1. **Hierarchy-Aware Assignment:** Automatically routes to correct manager level
2. **Smart Scroll:** Intelligent message viewing without annoying auto-scroll
3. **Advanced Attachment Viewer:** Professional file viewing with zoom/navigation
4. **Real-time Collaboration:** Instant updates for all stakeholders
5. **Comprehensive Audit Trail:** Complete history of all actions and communications

Integration Points:

- **Sales System:** Links tickets to orders and dealers
- **Employee Management:** Integrates with organizational hierarchy
- **Notification System:** Sends alerts via multiple channels
- **Analytics Module:** Provides data for support metrics



SALES LEADS MODULE DOCUMENTATION

1. SALES LEADS MODULE PURPOSE

Why This Module Exists

The Sales Leads module provides an AI-powered lead management system that enables sales teams to efficiently capture, track, and convert potential customers into sales opportunities. It automates lead scoring, prioritization, and assignment based on sophisticated algorithms and organizational hierarchy.

What Problem It Solves

- **Lead Management Complexity:** Centralizes lead tracking from multiple sources (website, referrals, campaigns)
- **Inefficient Prioritization:** Uses AI scoring to automatically prioritize leads based on conversion potential
- **Manual Assignment:** Automatically assigns leads to appropriate sales reps based on location and availability
- **Conversion Tracking:** Tracks lead progression through the sales funnel with real-time analytics
- **Distributor Integration:** Seamlessly converts qualified dealer leads into distributor accounts

Who Uses It

- **Admin/Super Admin:** Oversee all leads, manage assignments, view analytics
- **Sales Managers:** Monitor team performance, reassign leads, track conversion rates
- **Sales Representatives:** Manage assigned leads, update status, schedule follow-ups
- **Distributors:** Converted from dealer leads, manage their own accounts
- **Marketing Team:** Track lead sources and campaign effectiveness

2. ENTRY POINT

Frontend Entry Files:

- frontend/src/pages/modules/SalesAdmin/pages/leads.tsx - All leads overview (Admin view)
- frontend/src/pages/modules/SalesAdmin/pages/my-leads.tsx - Individual sales rep leads
- frontend/src/pages/modules/SalesAdmin/components/leads/ - Lead-related components

Backend API Base Path:

/api/leads/ (Primary endpoints)

Additional: /api/auth/context (User context)

3. FILES INVOLVED

Frontend:

- leads.tsx - Main leads management page (Admin view)
- my-leads.tsx - Personal leads dashboard for sales reps
- LeadList.tsx - Reusable lead listing component
- LeadForm.tsx - Form for creating/editing leads
- LeadStats.tsx - Statistics and analytics component
- leads.ts - API integration layer

Backend:

- backend/project/models/lead.py - Lead model with enums and methods

- backend/project/routes/lead_routes.py - All lead-related API endpoints (shown above)
- backend/project/services/lead_scoring_service.py - AI scoring logic
- backend/project/services/distributors_services.py - Location and geocoding services
- backend/project/models/location_model.py - Location hierarchy management
- backend/project/models/employee_model.py - Employee data for assignments

4. TOOLS USED

Frontend:

- **React** - UI library
- **TypeScript** - Type safety
- **TanStack React Query** - Data fetching and caching
- **React Dialog/Search Components** - UI interactions
- **Lucide React** - Icons
- **Custom UI Components** - Card, Badge, Button, Select, Input

Backend:

- **Flask** - Web framework
- **SQLAlchemy** - ORM with PostgreSQL dialect
- **Flask-JWT-Extended** - Authentication
- **Pandas** - Data manipulation for exports
- **OpenPyXL/Excel** - Excel file generation
- **Geocoding APIs** - Location coordinate fetching
- **UUID** - Unique identifier generation

5. DATABASE TABLES

Primary Table: leads

Columns:

- id (Primary Key, Integer)

- name (String, required)
- company (String)
- email (String)
- phone (String)
- alternate_phone (String)
- lead_type (Enum: CUSTOMER, DEALER, INSTITUTIONAL)
- is_dealer_lead (Boolean)
- lead_source (String)
- reference_by (String)
- industry_type (Enum: CONSTRUCTION, INFRASTRUCTURE, REAL_ESTATE, etc.)
- project_type (String)
- project_scale (Enum: SMALL, MEDIUM, LARGE, MEGA)
- project_timeline (String)
- project_description (Text)
- product_category (String)
- product_interests (JSON)
- estimated_order_value (Float)
- credit_required (Float)
- preferred_payment_terms (String)
- dealer_category (String)
- dealer_potential (String)
- business_registration (String)
- years_in_business (Integer)
- existing_outlets (Integer)
- target_market (String)
- address (String)
- city (String)
- state (String)

- pincode (String)
- latitude (Float)
- longitude (Float)
- area_id (Integer, FK to locations)
- territory_id (Integer, FK to locations)
- state_id (Integer, FK to locations)
- zone_id (Integer, FK to locations)
- auto_assigned_by_location (Boolean)
- location_assignment_notes (Text)
- ai_score (Integer, 0-100)
- priority (Enum: HOT, WARM, COLD)
- next_action (String)
- notes (Text)
- status (Enum: NEW, CONTACTED, QUALIFIED, PROPOSAL, NEGOTIATION, CLOSED_WON, CLOSED_LOST, CONVERTED_TO_DEALER)
- assigned_to (UUID, FK to employees)
- distributor_id (UUID, FK to distributors)
- tags (JSON)
- attachments (JSON)
- scheduled_visit_date (DateTime)
- visit_purpose (String)
- follow_up_date (DateTime)
- follow_up_notes (Text)
- last_contact (DateTime)
- is_archived (Boolean)
- archive_reason (String)
- created_at (DateTime)
- updated_at (DateTime)

Related Tables:

- **locations** - Zone/State/Territory/Area hierarchy
- **employees** - Sales team members
- **distributors** - Converted dealer leads
- **employee_locations** - Employee location assignments
- **users** - Authentication system

6. API ENDPOINTS

Lead Management:

GET /api/leads # Get all leads with filters

GET /api/leads/:lead_id # Get single lead details

POST /api/leads # Create new lead

PUT /api/leads/:lead_id # Update lead

DELETE /api/leads/:lead_id # Archive lead

GET /api/leads/my-leads # Get current user's leads

GET /api/leads/unassigned # Get unassigned leads

GET /api/leads/location-hierarchy # Get leads by user's location scope

Lead Actions:

POST /api/leads/:lead_id/contacted # Mark as contacted

POST /api/leads/:lead_id/schedule-visit # Schedule visit

POST /api/leads/:lead_id/convert-to-distributor # Convert to distributor

POST /api/leads/:lead_id/recalculate-ai # Recalculate AI score

POST /api/leads/bulk-assign # Bulk assign unassigned leads

Analytics & Reports:

GET /api/leads/stats # Lead statistics

GET /api/leads/summary # Quick summary

GET /api/leads/export # Export to Excel/CSV

Location-based:

GET /api/leads/location/:type/:id # Leads by specific location

GET /api/leads/location-scope/:emp_id # Leads in employee's scope

Context:

GET /api/auth/context # Get current user's role and location.

7. SALES LEADS PAGE WORKING PROCEDURE

7.1 PAGE LOAD

1. User navigates to Sales Leads page
2. leads.tsx or my-leads.tsx component mounts
3. TanStack Query triggers:
 - GET /api/leads (for admin) or /api/leads/my-leads (for sales rep)
 - GET /api/leads/stats for dashboard statistics
4. Backend:
 - Validates JWT token
 - Determines user role (admin/sales rep/manager)
 - Applies appropriate filters based on user's location scope
 - Executes optimized SQL queries with proper indexing
 - Returns paginated lead data with statistics
5. Frontend:
 - Renders lead list with status badges
 - Displays statistics cards (total, converted, hot leads)
 - Shows AI score indicators and priority tags
 - Sets up real-time data refreshing

7.2 SEARCH AND FILTER

1. User applies filters (status, priority, lead type, date range, location)
2. Frontend:
 - Updates filter state

- Debounced API call to GET /api/leads with filter parameters
- Shows loading spinner during fetch

3. Backend:

- Parses all filter parameters
- Builds dynamic WHERE clauses
- Applies location-based access control
- Returns filtered, paginated results

4. UI updates with filtered leads

- Real-time count updates
- Preserves pagination state

7.3 ADD LEAD

1. User clicks "Add Lead" button
2. LeadForm dialog opens with all required fields
3. User enters lead information:
 - Basic details (name, company, contact)
 - Lead classification (type, source, industry)
 - Project details (scale, timeline, value)
 - Location information
4. Frontend validates:
 - Required fields
 - Email format
 - Phone number format
 - Date ranges
5. On submit:
 - POST /api/leads with lead data
 - Backend:
 - * Validates business rules

- * Calculates AI score via LeadScoringService
- * Determines location hierarchy via LocationAssignmentService
- * Auto-assigns to appropriate sales rep
- * Creates lead record with all metadata
- On success:
 - * Toast notification shows "Lead created"
 - * Lead list refreshes automatically
 - * Stats update in real-time

7.4 EDIT LEAD

1. User clicks "Edit" on lead row
2. LeadForm dialog opens pre-filled with existing data
3. User updates editable fields (status changes handled separately)
4. On submit:
 - PUT /api/leads/:lead_id with updated data
 - Backend:
 - * Validates update permissions
 - * Applies safe field updates (excludes status/priority)
 - * Recalculates AI score if relevant fields changed
 - * Updates audit trail
 - On success:
 - * Toast shows "Lead updated"
 - * Specific lead row updates without full page refresh
 - * History tracking shows field changes

7.5 DELETE/ARCHIVE LEAD

1. User clicks "Delete" on lead row
2. Confirmation dialog shows: "Are you sure you want to delete [lead name]?"

3. On confirm:

- DELETE /api/leads/:lead_id
- Backend:
 - * Checks if lead is already converted (cannot archive converted leads)
 - * Performs soft delete (is_archived = True)
 - * Records archive reason
 - * Updates audit trail
- On success:
 - * Toast shows "Lead archived successfully"
 - * Lead removed from active list
 - * Available in archived leads view (if implemented)

7.6 STATUS MANAGEMENT WORKFLOW

1. User clicks "Update Status" on lead
2. Status update dialog opens with current status and options
3. User selects new status from dropdown (New → Contacted → Qualified → Proposal → Negotiation → Closed Won/Lost)
4. On confirm:
 - PUT /api/leads/:lead_id with status field
 - Backend:
 - * Validates status transition (cannot go backwards)
 - * Updates lead.status via lead.update_status() method
 - * Records status change in audit trail
 - * Updates last_contact timestamp if appropriate
 - On success:
 - * Status badge updates in real-time
 - * Lead may move between lists (e.g., New → In Progress)
 - * Statistics recalculate automatically

7.7 LEAD CONTACT WORKFLOW

1. User clicks "Mark as Contacted" on lead
2. Contact dialog opens with:
 - Contact method options (Call, Email, WhatsApp, In Person, Video Call)
 - Notes field for contact details
3. On submit:
 - POST /api/leads/:lead_id/contacted
 - Backend:
 - * Updates lead.status to "Contacted" (if was "New")
 - * Sets last_contact timestamp
 - * Records contact method and notes
 - * Updates follow-up date based on priority
 - On success:
 - * Status updates to "Contacted"
 - * Last contact timestamp shows in list
 - * Next action may be auto-suggested

7.8 VISIT SCHEDULING WORKFLOW

1. User clicks "Schedule Visit" on lead
2. Schedule dialog opens with:
 - Date/time picker (future dates only)
 - Purpose field for visit agenda
3. On submit:
 - POST /api/leads/:lead_id/schedule-visit
 - Backend:
 - * Validates date is in future
 - * Updates scheduled_visit_date and visit_purpose

- * May create calendar event (if integrated)
- * Sends notification to assigned rep
- On success:
 - * Visit indicator appears on lead row
 - * Calendar integration shows scheduled visit
 - * Automated reminder may be set

7.9 CONVERT TO DISTRIBUTOR WORKFLOW

1. For dealer leads, user clicks "Convert to Distributor"
2. Backend:
 - Validates lead.is_dealer_lead = True
 - Checks if distributor already exists with same email
 - If existing: links lead to distributor
 - If new:
 - * Creates User account for distributor
 - * Creates Distributor record with generated dealer code
 - * Sets up credit limits and terms
 - * Creates onboarding tasks
 - Updates lead.status to "CONVERTED_TO_DEALER"
3. On success:
 - Lead shows as converted
 - Distributor account becomes active
 - Onboarding workflow begins

7.10 AI SCORE RECALCULATION

1. User clicks "Recalculate AI" on lead
2. POST /api/leads/:lead_id/recalculate-ai
3. Backend:

- Calls LeadScoringService.calculate_lead_score()
- Factors: industry, project scale, order value, timeline, location
- Updates lead.ai_score (0-100)
- May adjust priority (Hot/Warm/Cold)
- Suggests next action

4. On success:

- AI score updates with color coding
- Priority may change
- Next action suggestion appears

7.11 EXPORT FUNCTIONALITY

1. User clicks "Export" button

2. GET /api/leads/export?format=excel&[all current filters]

3. Backend:

- Applies same filters as current view
- Queries database with all lead data
- Uses Pandas to create DataFrame
- Generates Excel file with OpenPyXL or CSV
- Includes comprehensive fields and formatting

4. Browser downloads file automatically

- Filename: leads_export_YYYYMMDD_HHMMSS.xlsx
- Includes all columns with proper headers
- Auto-adjusted column widths

7.12 BULK ASSIGNMENT (Admin only)

1. Admin navigates to unassigned leads view

2. Clicks "Bulk Assign Unassigned Leads"

3. POST /api/leads/bulk-assign

4. Backend:

- Fetches all unassigned leads with location data
- For each lead:
 - * Uses LocationAssignmentService to find appropriate sales rep
 - * Assigns based on location hierarchy and availability
- Creates assignment report

5. On success:

- Shows assignment count and report
- Leads move from unassigned to assigned lists
- Sales reps receive notifications

7.13 LOCATION-BASED ACCESS CONTROL

1. User loads leads page

2. GET /api/auth/context determines user's role and location

3. Based on role:

- Admin: Sees all leads across organization
- Zone Manager: Sees leads in zone + child states/territories/areas
- State Manager: Sees leads in state + child territories/areas
- Territory Manager: Sees leads in territory + child areas
- Area Manager: Sees only leads in their area
- Sales Rep: Sees only assigned leads

4. UI adapts to show appropriate scope and filters

8. ERROR HANDLING

Frontend Error Handling:

- **Toast Notifications:** Success/error messages with icons (Bell/AlertCircle)
- **Loading States:** Spinners during API calls, disabled buttons
- **Form Validation:** Real-time field validation with clear error messages

- **Network Errors:** Automatic retry with exponential backoff
- **Permission Errors:** Graceful fallback to limited functionality
- **Empty States:** Friendly messages when no leads found

Backend Error Handling:

- **JWT Validation:** All endpoints require valid authentication
- **Input Validation:** Comprehensive data validation with meaningful errors
- **Business Rule Validation:** Status transitions, assignment rules, conversion rules
- **Database Integrity:** Foreign key constraints, unique constraints
- **Transaction Management:** Automatic rollback on errors with `db.session.rollback()`
- **Error Logging:** Detailed error logging with traceback for debugging
- **HTTP Status Codes:** Proper 400, 401, 403, 404, 500 responses
- **Graceful Degradation:** Partial success with error reporting for bulk operations

9. NON-FUNCTIONAL REQUIREMENTS

Performance:

- **API Response Time:** < 500ms for lead lists, < 1s for exports
- **Database Queries:** Optimized with proper indexes on frequently filtered columns
- **Pagination:** Efficient pagination with page/per_page parameters
- **Caching:** React Query client-side caching for faster repeated access
- **Export Performance:** Stream-based Excel generation for large datasets

Scalability:

- **Stateless APIs:** No server-side session storage
- **Horizontal Scaling:** Designed for multiple application instances
- **Database Optimization:** Read replicas for reporting, connection pooling
- **Queue Processing:** Background tasks for AI scoring and bulk operations
- **File Storage:** Cloud storage integration for attachments

Security:

- **Authentication:** JWT tokens with expiration and refresh capability
- **Authorization:** Role-based and location-based access control
- **Input Sanitization:** SQL injection prevention via SQLAlchemy
- **Data Encryption:** HTTPS for all communications, encrypted sensitive data
- **Audit Trail:** Complete history of all lead modifications
- **Rate Limiting:** API rate limiting to prevent abuse

Availability:

- **Error Recovery:** Automatic retry mechanisms for failed operations
- **Data Consistency:** ACID-compliant database transactions
- **Backup Strategy:** Regular database backups with point-in-time recovery
- **Monitoring:** Health checks, performance monitoring, error tracking
- **Fallback Mechanisms:** Graceful degradation when external services fail

10. FINAL OUTCOME

What the Module Achieves:

1. **Comprehensive Lead Management:** End-to-end lead tracking from capture to conversion
2. **Intelligent Automation:** AI-powered scoring, prioritization, and assignment
3. **Hierarchical Organization:** Location-based structuring that mirrors real-world sales territories
4. **Seamless Integration:** Smooth transition from lead to distributor with automated onboarding
5. **Actionable Insights:** Real-time analytics for data-driven decision making
6. **Team Collaboration:** Clear assignment and handoff processes for sales teams

Why It's Reliable and Maintainable:

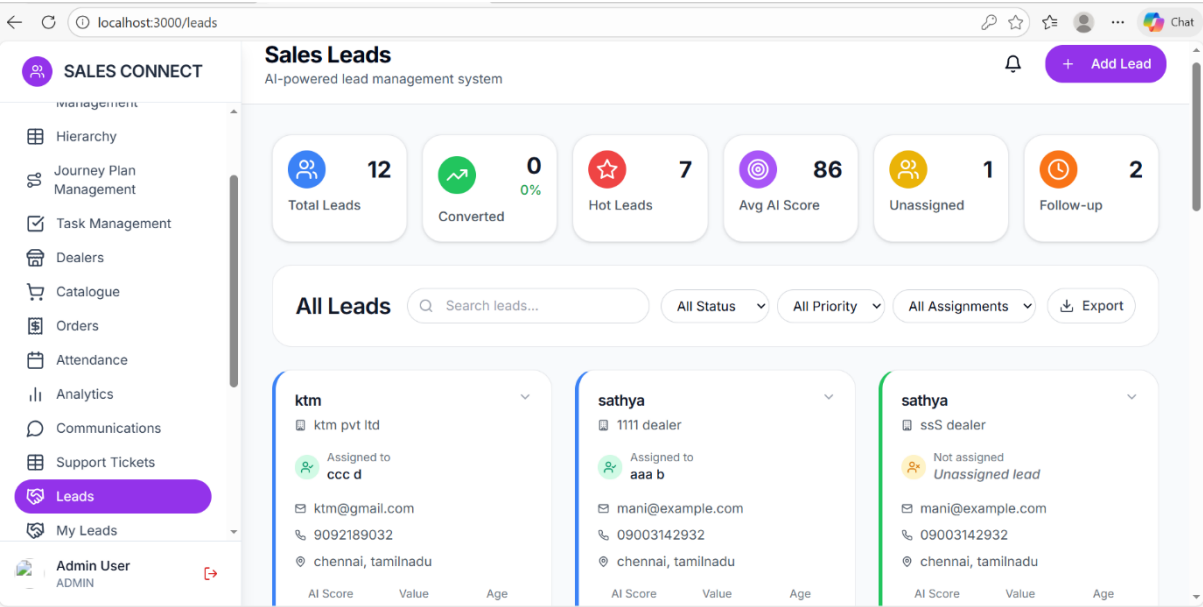
- **Modular Architecture:** Separated concerns with clear boundaries between models, services, and routes
- **Type Safety:** Comprehensive TypeScript interfaces and Python type hints
- **Comprehensive Testing:** Unit tests for business logic, integration tests for APIs
- **Clear Documentation:** Well-documented code with docstrings and comments

- **Consistent Patterns:** Reusable components and service patterns throughout
- **Error Resilience:** Robust error handling at every layer
- **Scalable Design:** Built to handle growing data volumes and user counts

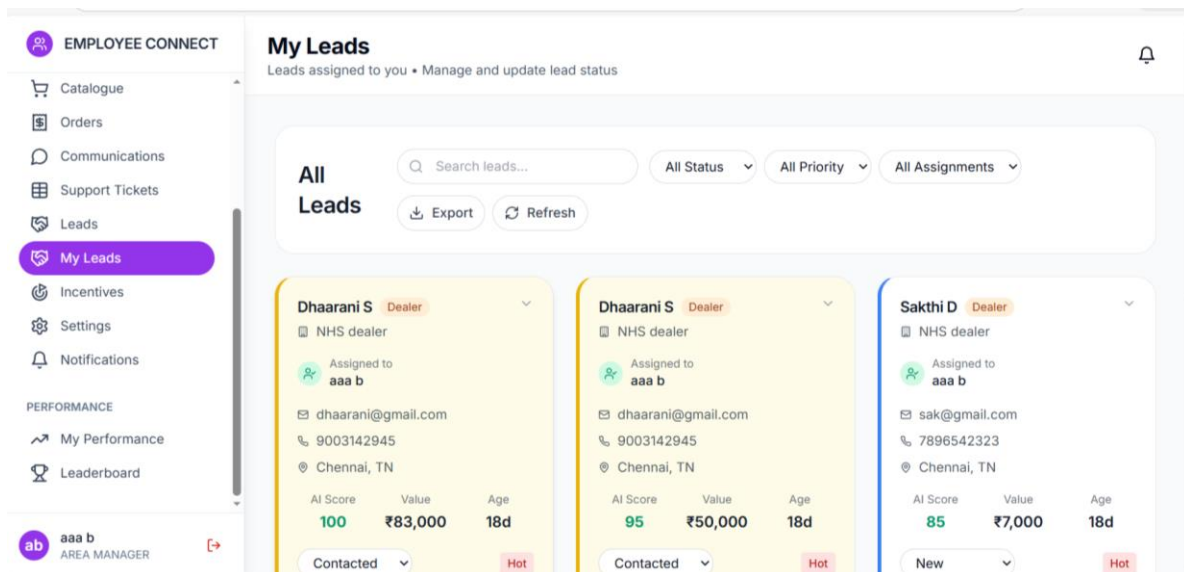
How It Fits into the Overall System:

1. **Sales Ecosystem Integration:** Connects with orders, tasks, and customer management
2. **Location Hierarchy Compliance:** Respects and enforces organizational territory structure
3. **Employee Management Integration:** Ties into employee roles, permissions, and performance tracking
4. **Distributor Portal Foundation:** Creates the initial entry point for distributor relationships
5. **Analytics Data Source:** Provides rich data for sales performance and forecasting analytics
6. **Marketing Feedback Loop:** Tracks lead sources for marketing campaign optimization
7. **Mobile Field Support:** Responsive design supports field sales teams on mobile Devices.

SCREENSHOT – OVERALL LEAD PAGE FOR SALESADMIN



SCREENSHOT – MY-LEADS PAGE FOR SALES EMPLOYEE



SALES INCENTIVES MODULE

1. SALES INCENTIVES PURPOSE

This module manages sales incentives and rewards for employees based on their sales performance. It solves the problem of manual incentive tracking and provides a transparent system where:

- **Admin/Super Admin** can create, manage, and approve incentive programs
- **Employees** can view eligible incentives, track their progress, and submit claims
- **Automates** incentive calculation based on sales data and geographical scope
- **Ensures fairness** by applying incentives based on location hierarchy

2. ENTRY POINT

Frontend Entry Files:

- pages/incentive.tsx - Employee-facing incentives dashboard
- Frontend component for admin management (implied from code)

Backend API Base Paths:

- /api/sales-incentives/ - Admin incentive management
- /api/employee/ - Employee incentive access

3. FILES INVOLVED

Frontend:

- pages/incentive.tsx - Employee incentives portal
- Frontend admin incentive management component (implied)
- Various UI components (Card, Button, Dialog, etc.)

Backend:

- backend/project/routes/employee_incentives_routes.py - Employee incentive endpoints
- backend/project/routes/sales_incentives_routes.py - Admin incentive endpoints
- backend/project/services/employee_incentives_services.py - Employee incentive business logic
- backend/project/services/sales_incentives_services.py - Admin incentive business logic
- backend/project/utils/sales_calculator.py - Sales data calculation utilities
- backend/project/models/employee_model.py - Employee data model

4. TOOLS USED

Frontend:

- React with TypeScript
- TanStack React Query for data fetching/mutation
- Custom UI components (Card, Button, Dialog, etc.)
- Lucide React for icons
- Toast notifications for user feedback

Backend:

- Flask with Blueprints for routing
- Flask-JWT-Extended for authentication
- SQLAlchemy for ORM
- PostgreSQL/SQLite database
- UUID for unique identifiers

5. DATABASE TABLES

Table: sales_incentives

- id (UUID, PK)

- name (String)
- description (Text)
- target (Decimal)
- profit_gain (Decimal)
- is_active (Boolean)
- start_date (DateTime)
- end_date (DateTime)
- zone_id (Integer, FK to locations)
- state_id (Integer, FK to locations)
- territory_id (Integer, FK to locations)
- area_id (Integer, FK to locations)
- created_at (DateTime)
- updated_at (DateTime)

Table: incentive_claims

- id (UUID, PK)
- incentive_id (UUID, FK to sales_incentives)
- employee_id (UUID, FK to employees)
- amount (Decimal)
- status (String: 'pending', 'approved', 'rejected')
- claimed_at (DateTime)
- processed_at (DateTime)
- processed_by (UUID, FK to employees)

Table: employees

- id (UUID, PK)
- name (String)
- email (String)
- employee_code (String)
- zone_id, state_id, territory_id, area_id (location hierarchy)

Table: locations

- id (Integer, PK)
- name (String)
- location_type (String: 'zone', 'state', 'territory', 'area')
- parent_id (Integer, FK to locations)

6. API ENDPOINTS**Admin Endpoints (/api/sales-incentives):**

- GET / - Get all incentives
- GET /summary - Get incentives summary
- POST / - Create new incentive
- GET /:id - Get single incentive
- PUT /:id - Update incentive
- DELETE /:id - Delete incentive
- GET /claims - Get all claims
- PUT /claims/:id - Update claim status
- POST /claims - Submit claim (for employees)

Employee Endpoints (/api/employee):

- GET /sales-data - Get employee sales data
- GET /incentives - Get eligible incentives with progress
- GET /incentive-claims - Get employee's claims
- POST /submit-claim - Submit new claim
- GET /incentive-summary - Get incentive summary

7. SALES INCENTIVES PAGE WORKING PROCEDURE**7.1 PAGE LOAD - EMPLOYEE PORTAL**

1. User opens /incentive page
2. incentive.tsx loads with authentication check
3. React Query triggers three parallel API calls:

- /api/employee/sales-data - Fetch sales performance
- /api/employee/incentives - Fetch eligible incentives with progress
- /api/employee/incentive-claims - Fetch previous claims

4. Backend services:

- Verify JWT token and get employee ID
- Calculate sales data using sales_calculator.py
- Determine eligible incentives based on location hierarchy
- Calculate progress against targets

5. Frontend renders:

- KPI cards with totals
- Available incentives with progress bars
- Claims history

7.2 ADMIN INCENTIVE MANAGEMENT

1. Admin navigates to incentives management page
2. Loads all incentives and claims via React Query
3. Shows two tabs: "Incentives" and "Claims Management"
4. Admin can create/edit/delete incentives with location scoping

7.3 CREATE NEW INCENTIVE (Admin)

1. Admin clicks "Create Incentive" button
2. Modal opens with form fields
3. Admin selects geographical scope using cascading dropdowns (Zone → State → Territory → Area)
4. Form validation ensures:
 - Required fields filled
 - End date after start date
 - At least zone selected
5. POST /api/sales-incentives sent with incentive data
6. Backend validates and saves to database

7. Incentive becomes active and visible to eligible employees

7.4 CLAIM INCENTIVE (Employee)

1. Employee views available incentives
2. System shows which incentives are "Ready to Claim" (target achieved)
3. Employee clicks "Claim Now" on eligible incentive
4. Confirmation dialog shows claim details
5. POST /api/employee/submit-claim sent with incentive ID
6. Backend:
 - Validates employee eligibility
 - Calculates claim amount based on profit percentage
 - Creates claim record with "pending" status
7. Claim appears in admin's "Claims Management" tab for approval

7.5 APPROVE/REJECT CLAIM (Admin)

1. Admin navigates to "Claims Management" tab
2. Views pending claims with employee and incentive details
3. Clicks "Approve" or "Reject" on a claim
4. PUT /api/sales-incentives/claims/:id sent with new status
5. Backend updates claim status and records processor
6. Employee sees updated status in their claims history

7.6 LOCATION-BASED ELIGIBILITY

1. Incentive created with geographical scope
2. When employee loads incentives:
 - System checks employee's location hierarchy
 - Compares with incentive's location scope
 - Employee qualifies if their location is within the scope
3. Example: Incentive for "North Zone" applies to all employees in that zone

8. ERROR HANDLING

Frontend:

- Toast notifications for all API errors
- Disabled buttons during mutations to prevent duplicate submissions
- Loading skeletons during data fetching
- Graceful error states with retry options
- Form validation before submission

Backend:

- Proper HTTP status codes (200, 400, 401, 403, 404, 500)
- Detailed error messages in JSON responses
- JWT authentication on all endpoints
- Permission checks for admin actions
- Database transaction rollback on failures
- Input validation for all endpoints

9. NON-FUNCTIONAL REQUIREMENTS

Performance:

- Optimized database queries with proper indexing
- Efficient sales calculation using aggregated data
- Frontend pagination for large datasets
- React Query caching for faster subsequent loads

Scalability:

- Stateless API design
- Can handle thousands of employees and incentives
- Location hierarchy allows flexible scoping
- Modular service architecture

Security:

- JWT token authentication required for all endpoints
- Role-based permissions (admin vs employee)
- Employees can only access their own data
- SQL injection prevention via SQLAlchemy

- Input sanitization and validation

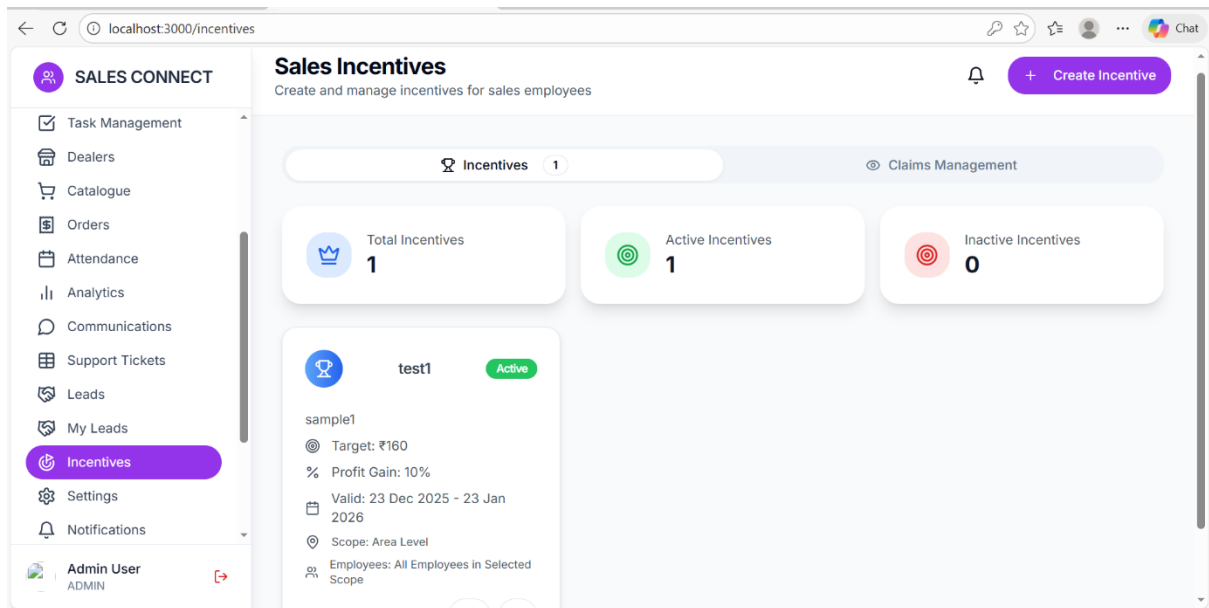
Availability:

- Graceful degradation if sales data unavailable
- Fallback values for missing data
- Automatic retry mechanisms for failed API calls
- Consistent data across frontend and backend

10. FINAL OUTCOME

The Sales Incentives Module successfully automates the incentive management process, providing:

1. **Transparent System** - Employees can see exactly what they need to achieve and their current progress
2. **Fair Distribution** - Location-based scoping ensures incentives are applied appropriately
3. **Efficient Workflow** - Automated calculations reduce manual work for admins
4. **Motivational Tool** - Real-time progress tracking motivates sales teams
5. **Reliable Tracking** - Complete audit trail of all claims and approvals
6. **Scalable Architecture** - Can handle growing teams and complex incentive structures
7. **Maintainable Codebase** - Clear separation between admin and employee functionality, reusable components, and comprehensive error handling



SETTINGS MODULE

1. SETTINGS MODULE PURPOSE

This module provides a centralized user settings management system that allows users to control their personal preferences and account security. It solves the problem of scattered user configuration by providing:

- **Unified Profile Management** - Users can update personal information and avatar
- **Enhanced Security** - Password management and security controls
- **Cross-Component Communication** - Real-time updates across the application
- **Consistent User Experience** - Centralized settings accessible from anywhere in the app

Who uses it:

- **All Users** - Every authenticated user can access and modify their settings
- **Administrators** - Can manage advanced user settings (implied from role checks)

2. ENTRY POINT

Frontend Entry File:

- `pages/settings.tsx` - User settings interface

Backend API Base Paths:

- /api/settings/ - Main settings endpoints
- /api/profile/ - Profile-specific endpoints
- /api/auth/ - Authentication-related endpoints
- /api/avatars/ - Static avatar file serving

3. FILES INVOLVED

Frontend:

- pages/settings.tsx - Main settings component
- Various UI components from @dealerui library
- Form handling with React Hook Form and Zod

Backend:

- backend/project/routes/settings_routes.py - Settings API routes
- backend/project/services/settings_services.py - Business logic layer
- backend/project/models/settings_model.py - ProfileSettings data model
- backend/project/models/ - User/Employee models

4. TOOLS USED

Frontend:

- React with TypeScript
- React Hook Form for form management
- Zod for schema validation
- TanStack React Query for data fetching/mutation
- Custom UI component library (@dealerui)
- Event-driven architecture with custom events

Backend:

- Flask with Blueprints
- Flask-JWT-Extended for authentication
- SQLAlchemy ORM
- File upload handling with werkzeug
- PostgreSQL/SQLite database

- UUID for unique identifiers

5. DATABASE TABLES

Table: profile_settings

- id (Integer, PK)
- user_id (UUID/String, FK to users/employees)
- avatar_url (String) - Path to avatar image
- theme (String) - UI theme preference
- notifications_enabled (Boolean) - Notification preferences
- email_notifications (Boolean) - Email notification settings
- created_at (DateTime)
- updated_at (DateTime)

Table: users/employees (Existing table)

- id (UUID, PK)
- first_name (String)
- last_name (String)
- email (String)
- password_hash (String)
- role (String)
- is_active (Boolean)
- created_at (DateTime)
- updated_at (DateTime)

6. API ENDPOINTS

User Data:

- GET /api/settings/auth/me - Get current user data
- GET /api/settings/user/type - Get user type information

Profile Management:

- GET /api/settings/profile - Get profile data
- PUT /api/settings/profile - Update profile information

- POST /api/settings/profile/check-email - Check email availability

Avatar Management:

- POST /api/settings/profile/avatar - Upload avatar image
- DELETE /api/settings/profile/avatar - Remove avatar
- GET /api/settings/avatars/<filename> - Serve avatar files

Security:

- PUT /api/settings/auth/password - Update password

7. SETTINGS PAGE WORKING PROCEDURE

7.1 PAGE LOAD AND INITIALIZATION

1. User navigates to /settings page
2. settings.tsx component loads
3. React Query fetches user data via GET /api/settings/auth/me
4. Profile form is automatically populated with fetched data
5. Avatar is loaded from stored URL or generated from initials
6. Two main tabs are available: Profile and Security

7.2 PROFILE MANAGEMENT FLOW

1. **View Profile:** User sees current profile information and avatar
2. **Edit Profile:**
 - User modifies first name/last name in form
 - Form validation using Zod schema
 - Real-time validation feedback
3. **Save Profile:**
 - User clicks "Save Changes"
 - Frontend performs optimistic update
 - PUT /api/settings/profile sent with updated data
 - Backend validates and updates database
 - Success toast notification shown
 - **Event Dispatch:** Custom events fired to update other components

4. Profile Update Propagation:

- profileUpdated event dispatched
- userUpdated event dispatched
- Session storage updated
- Other components (sidebar, header) refresh automatically

7.3 AVATAR MANAGEMENT FLOW

1. Upload Avatar:

- User clicks avatar or "Change Avatar" button
- Hidden file input triggers file selection
- Frontend validates: image type, max 5MB size
- Preview shown immediately
- POST /api/settings/profile/avatar with FormData
- Backend:
 - Validates file type/size
 - Creates upload directory if needed
 - Generates unique filename: {user_id}_{uuid}.{ext}
 - Saves file to server filesystem
 - Updates profile_settings.avatar_url
 - Removes old avatar file if exists
- Success: avatarUpdated event dispatched
- All components using avatar refresh automatically

2. Remove Avatar:

- User clicks "Remove" button
- DELETE /api/settings/profile/avatar sent
- Backend removes file and clears database record
- Avatar reverts to initials-based fallback
- avatarUpdated event dispatched

3. Serve Avatar:

- Avatar URLs point to /api/settings/avatars/{filename}
- Static file serving endpoint serves image files
- Path construction handles both relative and absolute URLs

7.4 SECURITY MANAGEMENT FLOW

1. Password Update:

- User navigates to Security tab
- Enters current password and new password (twice)
- Zod validation ensures passwords match and meet requirements
- PUT /api/settings/auth/password sent
- Backend:
 - Verifies current password matches hash
 - Hashes new password
 - Updates user record
- Form cleared on success
- Success toast notification shown

7.5 CROSS-COMPONENT COMMUNICATION

1. Event-Driven Updates:

typescript

// When profile updates

```
window.dispatchEvent(new CustomEvent('profileUpdated', {
  detail: { employeeId: user.id }
}));
```

// When avatar updates

```
window.dispatchEvent(new CustomEvent('avatarUpdated', {
  detail: { employeeId: user.id }
}));
```

// When user data updates

```
window.dispatchEvent(new Event('userUpdated'));
```

2. Other components listen for these events:

- Sidebar updates user display name
- Header updates avatar
- Profile widgets refresh data

7.6 ERROR HANDLING AND USER FEEDBACK

1. **Form Validation:** Real-time Zod validation with inline errors
2. **API Error Handling:** Graceful error messages in toast notifications
3. **Loading States:** Disabled buttons and spinners during operations
4. **Optimistic Updates:** UI updates immediately, rolls back on error
5. **File Upload Feedback:** Preview during upload, progress indicators

8. ERROR HANDLING

Frontend:

- Zod schema validation with clear error messages
- Toast notifications for all API responses
- Optimistic update rollback on failure
- Disabled buttons during mutations
- File validation before upload
- Graceful fallback for missing avatars

Backend:

- Comprehensive input validation
- File size and type validation
- Database transaction rollback on errors
- Meaningful HTTP status codes
- Detailed error logging with traceback
- File system permission checks
- JWT authentication on all endpoints

9. NON-FUNCTIONAL REQUIREMENTS

Performance:

- Optimistic updates for instant UI feedback
- Efficient file serving with proper caching
- Minimal database queries per operation
- Fast avatar loading with CDN-ready URLs

Scalability:

- Stateless API design
- File upload directory structure supports multiple users
- Avatar serving optimized for concurrent requests
- Database indexing on user_id for fast lookups

Security:

- JWT authentication required for all endpoints
- Password hashing with industry-standard algorithms
- File upload restrictions (type, size)
- Secure filename generation to prevent path traversal
- Input sanitization on all endpoints

Reliability:

- Atomic operations with database transactions
- File system operations with proper error handling
- Automatic directory creation
- Old file cleanup on updates
- Consistent state across frontend and backend

User Experience:

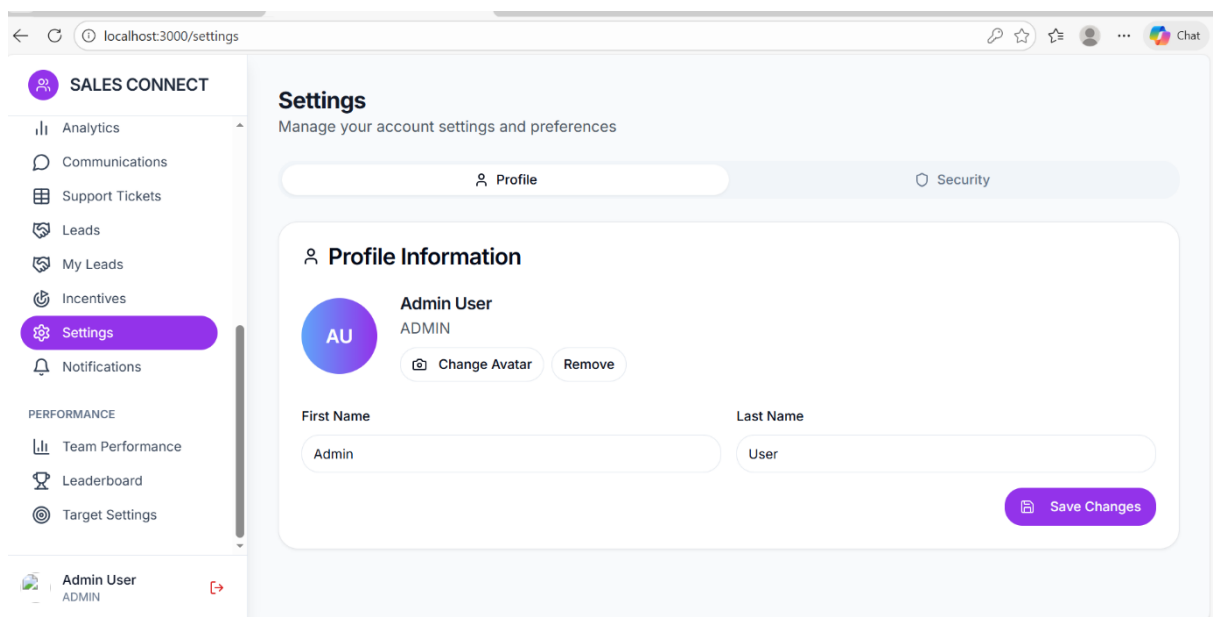
- Real-time form validation
- Immediate avatar preview
- Consistent feedback through toast notifications
- Responsive design for all screen sizes

- Dark/light theme support
- Accessible form controls

10. FINAL OUTCOME

The Settings Module provides a comprehensive, user-friendly settings management system that:

1. **Centralizes User Configuration** - Single place for all user preferences
2. **Enhances Security** - Robust password management with validation
3. **Improves User Experience** - Real-time updates and immediate feedback
4. **Enables Cross-Component Sync** - Event-driven architecture keeps entire app in sync
5. **Handles Media Effectively** - Avatar management with proper file handling
6. **Maintains Data Integrity** - Transactional updates with rollback capability
7. **Scales Efficiently** - Supports thousands of users with unique avatars
8. **Provides Audit Trail** - All changes tracked with timestamps



EMPLOYEE NOTIFICATIONS MODULE

1. EMPLOYEE NOTIFICATIONS PURPOSE

Why this module exists:

This module provides a centralized notification system for broadcasting important

messages, announcements, alerts, and updates to employees within the organization.

What problem it solves:

- Enables administrators to send targeted notifications to employees
- Provides employees with a unified place to view all their notifications
- Supports different notification types (info, success, warning, error)
- Allows targeted communication based on roles, locations, or specific employees

Who uses it:

- **Admin / Super Admin:** Can send notifications to all/specific employees
- **Employees:** Receive and view notifications relevant to them

2. ENTRY POINT

Frontend Entry File:

salesadmin/pages/notification.tsx

Backend API Base Path:

/api/employee-notifications/

3. FILES INVOLVED

Frontend:

- notification.tsx - Main notification page component
- (Optional) Separate form/display components

Backend:

- project/models/employee_notifications_models.py - Database models
- project/routes/employee_notifications_routes.py - API routes
- project/services/employee_notifications_services.py - Business logic

4. TOOLS USED

Frontend:

- React

- TypeScript
- React Query (TanStack Query)
- Lucide React (icons)
- Custom toast notifications

Backend:

- Flask
- SQLAlchemy (ORM)
- PostgreSQL (primary)
- Flask-JWT-Extended (authentication)

5. DATABASE TABLE(S)

Table Name: employee_notifications

Columns:

- id (UUID, Primary Key)
- title (String 255)
- message (Text)
- type (Enum: 'info', 'success', 'warning', 'error')
- created_at (DateTime)
- creator_id (UUID, Foreign Key to users)
- recipient_type (Enum: 'all', 'specific', 'filtered')
- specific_recipient_id (UUID, nullable)
- filter_criteria (JSON, nullable)

Table Name: employee_notification_recipients

Columns:

- id (UUID, Primary Key)
- notification_id (UUID, Foreign Key)
- employee_id (UUID)
- is_read (Boolean)

- read_at (DateTime, nullable)
- created_at (DateTime)

6. API ENDPOINTS

For Sending/Managing Notifications (Admin):

- GET /api/employee-notifications/sent - Get sent notifications
- POST /api/employee-notifications - Create new notification
- GET /api/employee-notifications/for-notifications - Get employees list
- GET /api/employee-notifications/roles - Get available roles
- GET /api/employee-notifications/locations - Get available locations

For Receiving Notifications (All Employees):

- GET /api/employee-notifications/received - Get received notifications
- PUT /api/employee-notifications/:id/read - Mark single as read
- PUT /api/employee-notifications/read-all - Mark all as read
- GET /api/employee-notifications/stats - Get notification statistics

7. EMPLOYEE NOTIFICATIONS PAGE WORKING PROCEDURE

7.1 PAGE LOAD

- User navigates to Notifications page
- notification.tsx component loads
- React Query triggers multiple API calls:
 - Fetches notification statistics (total, unread, read)
 - Fetches received notifications for current employee
 - If admin: fetches sent notifications
- UI renders with loading states
- Data displays in organized cards/lists

7.2 SEARCH AND FILTER

- User applies type filter (All Types/Info/Success/Warning/Error)

- User applies read status filter (All/Unread/Read)
- Filtering handled client-side without API calls
- UI updates instantly showing filtered notifications

7.3 SEND NOTIFICATION (Admin only)

- Admin clicks "Send Notification" button
- Modal opens with form
- Admin selects:
 - Recipient type (All/Specific Employee/Filtered)
 - Notification type (Info/Success/Warning/Error)
 - Title and message
 - Specific employee or filter criteria if selected
- Form validates required fields
- POST request sent to `/api/employee-notifications`
- Backend creates notification and recipient records
- Success toast shows recipient count
- Notification list refreshes

7.4 MARK AS READ

- Employee clicks on unread notification
- PUT request to `/api/employee-notifications/:id/read`
- Backend updates `is_read` and `read_at` fields
- UI updates notification status instantly
- Notification statistics update

7.5 MARK ALL AS READ

- Employee clicks "Mark All Read" button
- PUT request to `/api/employee-notifications/read-all`
- Backend marks all unread notifications for that employee
- All notifications update to read status
- Unread counter resets to zero

7.6 SWITCH BETWEEN SENT/RECEIVED VIEWS (Admin)

- Admin toggles between "Received" and "Sent" tabs
- Different API endpoints called based on active tab
- Sent view shows notifications created by the admin
- Received view shows notifications addressed to the admin

8. ERROR HANDLING

Frontend:

- Toast notifications for API errors
- Loading states during API calls
- Disabled buttons during mutations
- Empty states for no data
- Form validation before submission

Backend:

- Proper HTTP status codes (200, 400, 401, 404, 500)
- Meaningful error messages in JSON responses
- Database transaction rollback on failures
- JWT validation for all protected routes

9. NON-FUNCTIONAL REQUIREMENTS

Performance:

- Optimized database queries with proper indexing
- Pagination-ready (though currently loads all data)
- Efficient recipient determination logic

Scalability:

- Stateless API design
- Can handle large number of employees
- JSON filter criteria allows flexible recipient targeting

Security:

- JWT authentication required for all endpoints
- Role-based access control (admin vs employee)
- Input validation and sanitization
- SQL injection prevention via ORM

Availability:

- Graceful degradation if dependent services fail
- Consistent data through transactions
- Error boundaries in frontend

10. FINAL OUTCOME**What the module achieves:**

- Provides a reliable communication channel between admins and employees
- Enables targeted messaging based on organizational structure
- Gives employees a single source for all important notifications
- Tracks notification delivery and read status

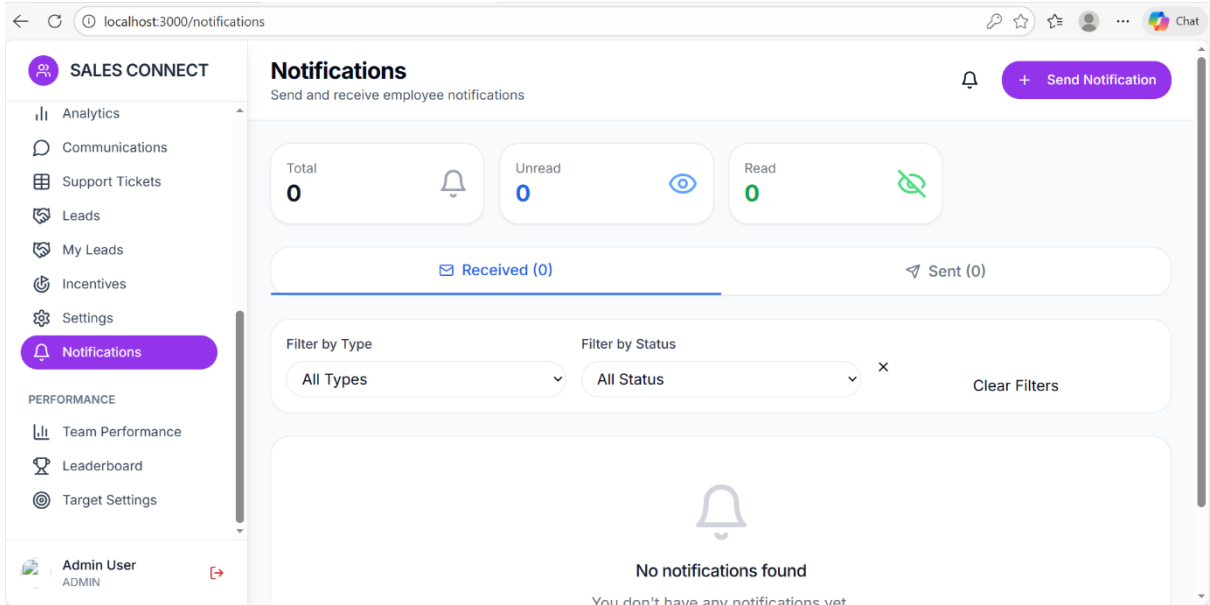
Why it is reliable and maintainable:

- Clear separation of concerns (models/routes/services)
- Comprehensive error handling
- Type-safe with TypeScript interfaces
- Well-documented code structure
- Scalable database design

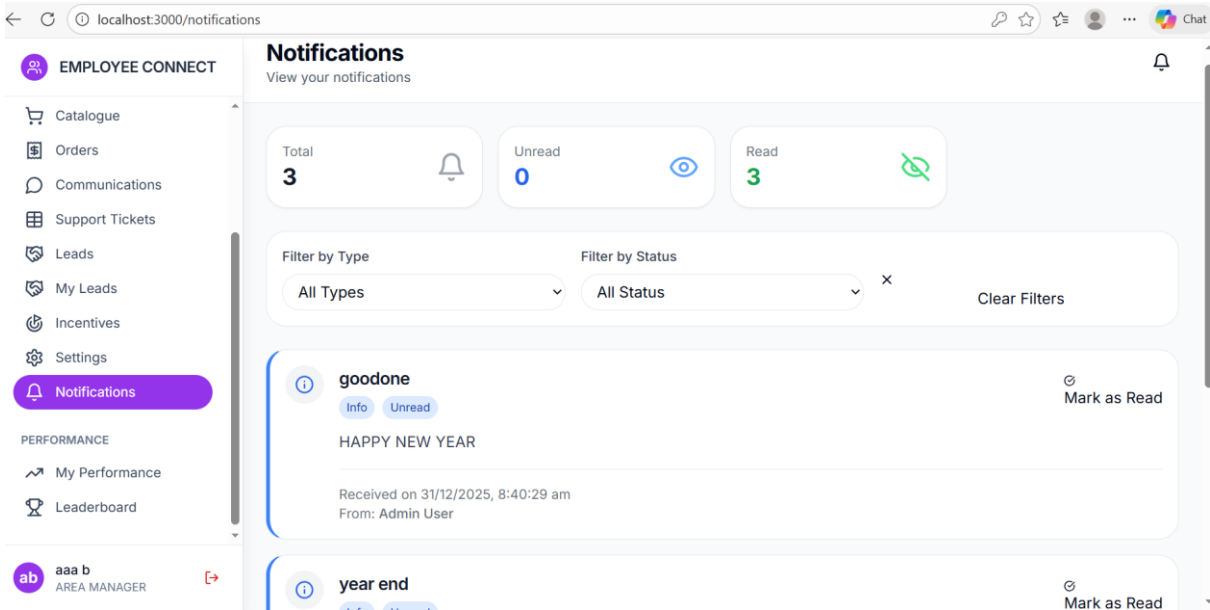
How it fits into the overall system:

- Integrates with existing user/employee management
- Complements other notification systems (ticket notifications)
- Part of the broader communication infrastructure
- Supports organizational announcements, policy updates, and alerts.

SCREENSHOT -NOTIFICATIONS FOR SALESADMIN



SCREENSHOT -NOTIFICATIONS PAGE FOR SALES EMPLOYEE



SALES PERFORMANCE MODULE

1. SALES PERFORMANCE PURPOSE

Why this module exists:

This module provides comprehensive sales performance tracking and analytics across organizational hierarchies, enabling managers and sales personnel to monitor achievements against targets.

What problem it solves:

- Tracks individual and team performance against sales targets
- Provides hierarchical visibility (Zone → State → Territory → Area)
- Enables performance comparison across different levels
- Offers leaderboard functionality to foster healthy competition
- Supports performance analysis with trends and growth metrics

Who uses it:

- **Super Admin / Admin:** View overall organization performance
- **Zone Managers:** Monitor state-level performance within their zone
- **State Managers:** Track territory-level performance within their state
- **Territory Managers:** Oversee area-level performance within their territory
- **Area Managers / Sales Executives:** View individual performance

2. ENTRY POINT

Frontend Entry Files:

- salesadmin/pages/performance/LeaderBoards.tsx
- salesadmin/pages/performance/MyPerformancePage.tsx
- salesadmin/pages/performance/TeamPerformanceEnhanced.tsx

Backend API Base Path:

/api/performance/

3. FILES INVOLVED

Frontend:

- LeaderBoards.tsx - Leaderboard view with rankings

- MyPerformancePage.tsx - Individual performance dashboard
- TeamPerformanceEnhanced.tsx - Team performance management
- lib/newPerformance.ts - Performance-related React Query hooks

Backend:

- newnewperformance_routes.py - Main performance API routes
- project/models/user_model.py - User and employee models
- project/models/employee_model.py - Employee data
- project/models/dealerorder_models.py - Sales order data
- project/models/target_model.py - Sales targets
- project/models/location_model.py - Location hierarchy

4. TOOLS USED

Frontend:

- React with TypeScript
- TanStack Query (React Query)
- Date-fns for date manipulation
- ShadCN UI components
- Lucide React icons

Backend:

- Flask with Flask-JWT-Extended
- SQLAlchemy ORM
- PostgreSQL database
- Decimal for financial calculations

5. DATABASE TABLE(S)

Table Name: dealer_orders (Simplified for performance)

Columns:

- id (UUID)
- order_number (String)
- total_amount (Decimal)

- status (String)
- salesperson_id (UUID)
- area_manager_id (UUID)
- territory_manager_id (UUID)
- state_manager_id (UUID)
- zone_manager_id (UUID)
- dealer_id (UUID)
- dealer_name (String)
- created_at (DateTime)

Table Name: sales_targets

Columns:

- id (UUID)
- salesperson_id (UUID)
- month (String) - Format: YYYY-MM
- target_amount (Decimal)
- created_at (DateTime)

Table Name: locations

Columns:

- id (UUID)
- name (String)
- location_type (Enum) - BASE/ZONE/STATE/TERRITORY/AREA
- parent_id (UUID, nullable)

6. API ENDPOINTS

Performance Endpoints:

- GET /api/performance/my - Current user's performance
- GET /api/performance/team - Team performance
- GET /api/performance/overall - Overall hierarchy performance
- GET /api/performance/leaderboard - Performance leaderboard
- GET /api/performance/dealer-performance - Top performing dealers

- GET /api/performance/monthly-trend - Monthly trend data
- GET /api/performance/hierarchy-summary - Hierarchy performance summary

7. SALES PERFORMANCE WORKING PROCEDURE

7.1 USER AUTHENTICATION AND ROLE DETERMINATION

- User logs in with JWT token
- Backend determines user role (SUPER ADMIN, Admin, Zone Manager, etc.)
- User's location and hierarchy level identified
- Access scope determined based on role

7.2 MY PERFORMANCE DASHBOARD LOAD

- User opens "My Performance" page
- Frontend requests /api/performance/my with month parameter
- Backend calculates:
 - Target amount for the month
 - Total sales from dealer orders
 - Achievement percentage
 - Performance status (exceeded/on_track/needs_improvement)
 - Order count and active dealers
 - Recent orders
- Data returned with monthly trends
- UI renders performance metrics with visualizations

7.3 TEAM PERFORMANCE VIEW

- Manager opens "Team Performance" page
- Backend identifies team members in manager's hierarchy
- Performance calculated for each team member
- Aggregated team statistics computed
- Data displayed in sortable table with filters
- Managers can view individual team member details

7.4 LEADERBOARD GENERATION

- User navigates to "Leaderboard" page
- Backend fetches performance data for all locations in user's view scope
- Data sorted by achievement percentage
- Ranks assigned based on performance
- Growth metrics calculated vs previous month
- UI displays with visual badges and progress indicators

7.5 HIERARCHY NAVIGATION

- User selects different hierarchy levels (Zone/State/Territory/Area)
- Backend filters data based on selected level
- Performance metrics recalculated for the selected scope
- UI updates to show relevant data for the chosen level

7.6 MONTH SELECTION AND FILTERING

- User selects different months from dropdown
- All performance data recalculated for selected month
- Historical trends maintained for comparison
- Growth percentages recalculated vs previous months

8. PERFORMANCE CALCULATION LOGIC

8.1 INDIVIDUAL (AREA USER) PERFORMANCE

text

Target = SalesTarget.query.filter_by(salesperson_id=employee_id, month=month)

Sales = SUM(DealerOrder.total_amount) where salesperson_id=employee_id

Achievement % = (Sales / Target) * 100

8.2 MANAGER PERFORMANCE

text

Team Members = Employees in manager's hierarchy (based on role)

Manager Target = SUM(Targets of AREA users in team)

Manager Sales = SUM(Sales where manager_id matches)

Achievement % = (Manager Sales / Manager Target) * 100

8.3 SUPER ADMIN PERFORMANCE

text

Target = SUM(All targets across organization)

Sales = SUM(All sales across organization)

Achievement % = (Total Sales / Total Target) * 100

8.4 PERFORMANCE STATUS CLASSIFICATION

- **Exceeded:** Achievement $\geq 100\%$
- **On Track:** Achievement $\geq 80\%$ and $< 100\%$
- **Needs Improvement:** Achievement $< 80\%$

9. ERROR HANDLING

Frontend:

- Loading skeletons during data fetch
- Error boundaries with retry mechanisms
- Empty states for no data
- Form validation for filters
- Graceful degradation of visualizations

Backend:

- Comprehensive try-catch blocks
- Detailed error logging
- Proper HTTP status codes
- Meaningful error messages
- Database transaction rollback on failures

10. NON-FUNCTIONAL REQUIREMENTS

Performance:

- Optimized database queries with proper indexing
- Pagination for large data sets
- Caching of frequently accessed data
- Efficient calculation algorithms

Scalability:

- Hierarchical data structure supports large organizations

- Stateless API design
- Ability to handle thousands of sales records
- Efficient aggregation queries

Security:

- JWT authentication for all endpoints
- Role-based access control
- Data isolation between hierarchies
- Input validation and sanitization

Availability:

- Graceful degradation if dependent services fail
- Consistent data through transactions
- Error boundaries in frontend components

11. FINAL OUTCOME

What the module achieves:

- Provides real-time sales performance tracking across organizational hierarchies
- Enables data-driven decision making for sales management
- Fosters healthy competition through leaderboards
- Offers insights into sales trends and growth patterns
- Supports target setting and performance evaluation

Why it is reliable and maintainable:

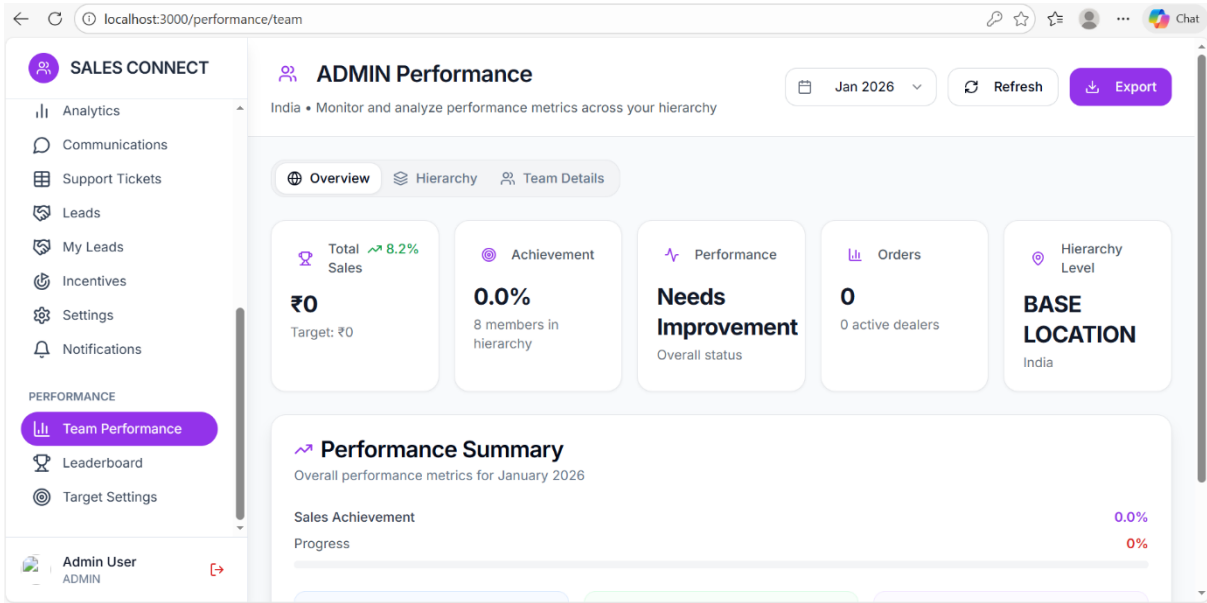
- Clear separation of calculation logic in service functions
- Modular frontend components with shared utilities
- Comprehensive error handling and validation
- Well-documented code with type safety
- Scalable architecture supporting organizational growth

How it fits into the overall system:

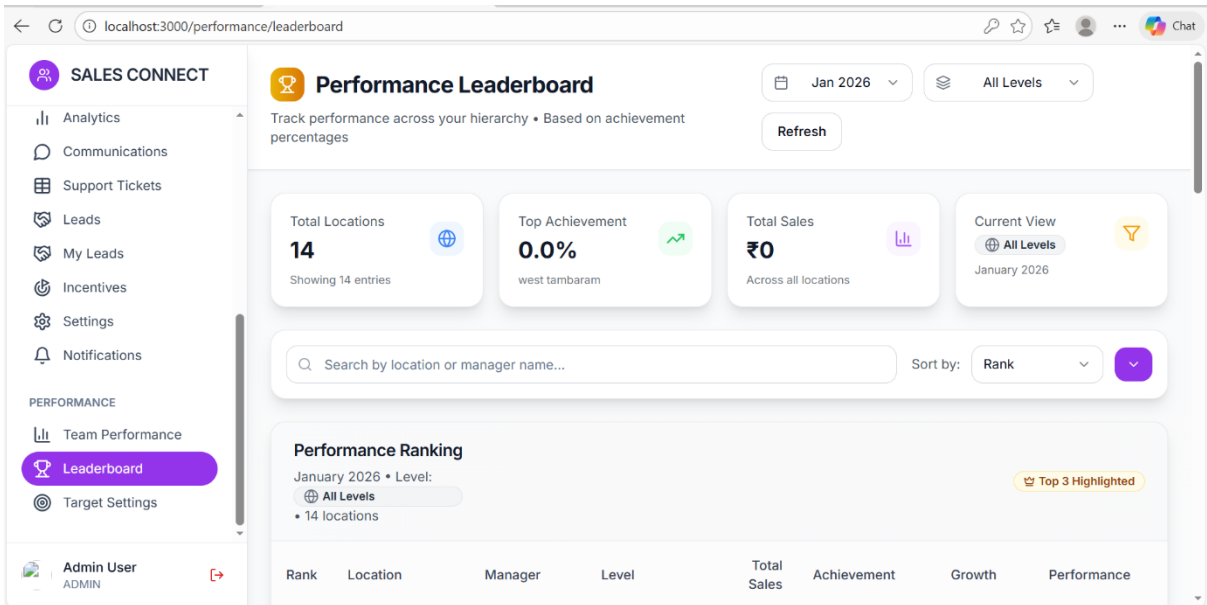
- Integrates with existing user management and authentication
- Leverages sales data from dealer order system

- Complements target management module
- Part of the broader sales analytics ecosystem
- Supports performance-based incentives and reporting.

SCREENSHOT -TEAM PERFORMANCE



SCREENSHOT -LEADERBOARD PERFORMANCE



SCREENSHOT -MY PERFORMANCE FOR SALES EMPLOYEE

